



Universidad
de Huelva

Fundamentos de Computadores

1º Curso del Grado en Ingeniería Informática

Manual de introducción al uso del software Vivado de Xilinx

Tarjeta de desarrollo ZedBoard

Índice

1. Introducción.	1
2. Creación de un nuevo proyecto.	2
3. Edición del fichero fuente.	9
4. Simulación del sistema.	10
5. Síntesis del sistema.	16
6. Implementación del sistema.	18
7. Programación de la tarjeta de desarrollo.	22

Índice de figuras

Figura 1 . Vista superior de la tarjeta de desarrollo ZedBoard.....	1
Figura 2 . Opciones para ejecutar el software Vivado desde el sistema operativo Windows.....	2
Figura 3 . Ventana inicial del software Vivado.	3
Figura 4 . Ventana inicial de la opción Create Project.....	3
Figura 5 . Indicación del nombre y la ubicación del proyecto a crear.	4
Figura 6 . Elección del tipo de proyecto a crear.	4
Figura 7 . Adición al proyecto de ficheros fuente ya existentes o creación de nuevos ficheros.	5
Figura 8 . Asignación del nombre al fichero fuente a crear.....	5
Figura 9 . Presentación del fichero fuente a crear en la lista de ficheros fuente del proyecto.....	6
Figura 10 . Ventana para la adición de ficheros de restricciones al proyecto.	6
Figura 11 . Configuración de la ventana <i>Default Part</i> para el empleo de la tarjeta de desarrollo ZedBoard.....	7
Figura 12 . Resumen de las características del proyecto creado.....	7
Figura 13 . Inicialización de la creación del proyecto en Vivado.	7
Figura 14 . Configuración de las características generales del módulo fuente VHDL.	8
Figura 15 . Pantalla principal del software Vivado correspondiente al proyecto.....	9
Figura 16 . Contenido del fichero fuente VHDL editado para incluir las ecuaciones del ejemplo.....	10
Figura 17 . Selección del tipo simulación (<i>VHDL Test Bench</i>) para el nuevo fichero fuente.	10
Figura 18 . Adición al proyecto de ficheros fuente de simulación o creación de nuevos ficheros.....	11
Figura 19 . Asignación del nombre al nuevo fichero fuente de simulación.	11
Figura 20 . Presentación del fichero de simulación a crear en la lista de ficheros fuente del proyecto	12
Figura 21 . Configuración de las características del fichero de simulación.	12
Figura 22 . Localización del fichero de simulación en la ventana principal de Vivado.	13
Figura 23 . Ventana de edición del fichero de simulación.....	13
Figura 24 . Ejecución de la simulación.....	15
Figura 25 . Inicio de la simulación del circuito en Vivado.....	15
Figura 26 . Cronograma resultante de la simulación del circuito del ejemplo.	16
Figura 27 . Ejecución de la síntesis del diseño.....	17
Figura 28 . Confirmación de la ejecución de la síntesis del diseño.	17
Figura 29 . Síntesis del diseño en proceso de ejecución.	17
Figura 30 . Comunicación de la finalización satisfactoria del proceso de síntesis.....	18
Figura 31 . Selección del tipo fichero de restricciones para el nuevo fichero fuente.	19
Figura 32 . Adición al proyecto de un fichero de restricciones o creación de uno nuevo.....	19
Figura 33 . Presentación del fichero de restricciones de la tarjeta ZedBoard en la lista de ficheros del proyecto....	20
Figura 34 . Presentación del fichero de restricciones de la tarjeta ZedBoard en la pantalla principal de Vivado.	20
Figura 35 . Modo de conexión de los terminales del circuito en la tarjeta de desarrollo ZedBoard.....	21
Figura 36 . Ejecución de la implementación del diseño.	22
Figura 37 . Comunicación de la finalización satisfactoria del proceso de implementación.	22
Figura 38 . Generación del fichero de programación de la tarjeta (<i>Bitstream</i>).....	23
Figura 39 . Confirmación de la generación del fichero de programación.	23
Figura 40 . Comunicación de la creación satisfactoria del fichero de restricciones.....	24
Figura 41 . Apertura del gestor del hardware.	24
Figura 42 . Establecimiento de la conexión con la tarjeta de desarrollo.....	25
Figura 43 . Programación de la tarjeta de desarrollo.	25
Figura 44 . Fichero a emplear para la programación de la tarjeta de desarrollo.	26

1. Introducción.

En el presente manual se indica la secuencia de pasos a ejecutar para el desarrollo de un proyecto, basado en el lenguaje VHDL, mediante el empleo del paquete de software Vivado de Xilinx.

Para ilustrar el proceso nos valdremos de un ejemplo sencillo, consistente en la descripción de dos funciones combinacionales F1 y F2.

En primer lugar explicaremos la forma de generar un nuevo proyecto, luego añadiremos al mismo un módulo fuente VHDL donde definiremos directamente las ecuaciones booleanas de las funciones a implementar, a continuación simularemos el circuito resultante y finalmente llevaremos a cabo las tareas necesarias para poder programar una tarjeta de desarrollo con el circuito obtenido y comprobar su funcionamiento sobre un sistema real.

Para la implementación física en el laboratorio de los proyectos creados usaremos la tarjeta de desarrollo **ZedBoard**, perteneciente a la familia ZINQ, que está basada en la FPGA **XC7Z020-CLG484-1**. En la Figura 1 se muestra una vista superior de la tarjeta de desarrollo ZedBoard sobre la cual se han destacado los principales elementos que la integran.

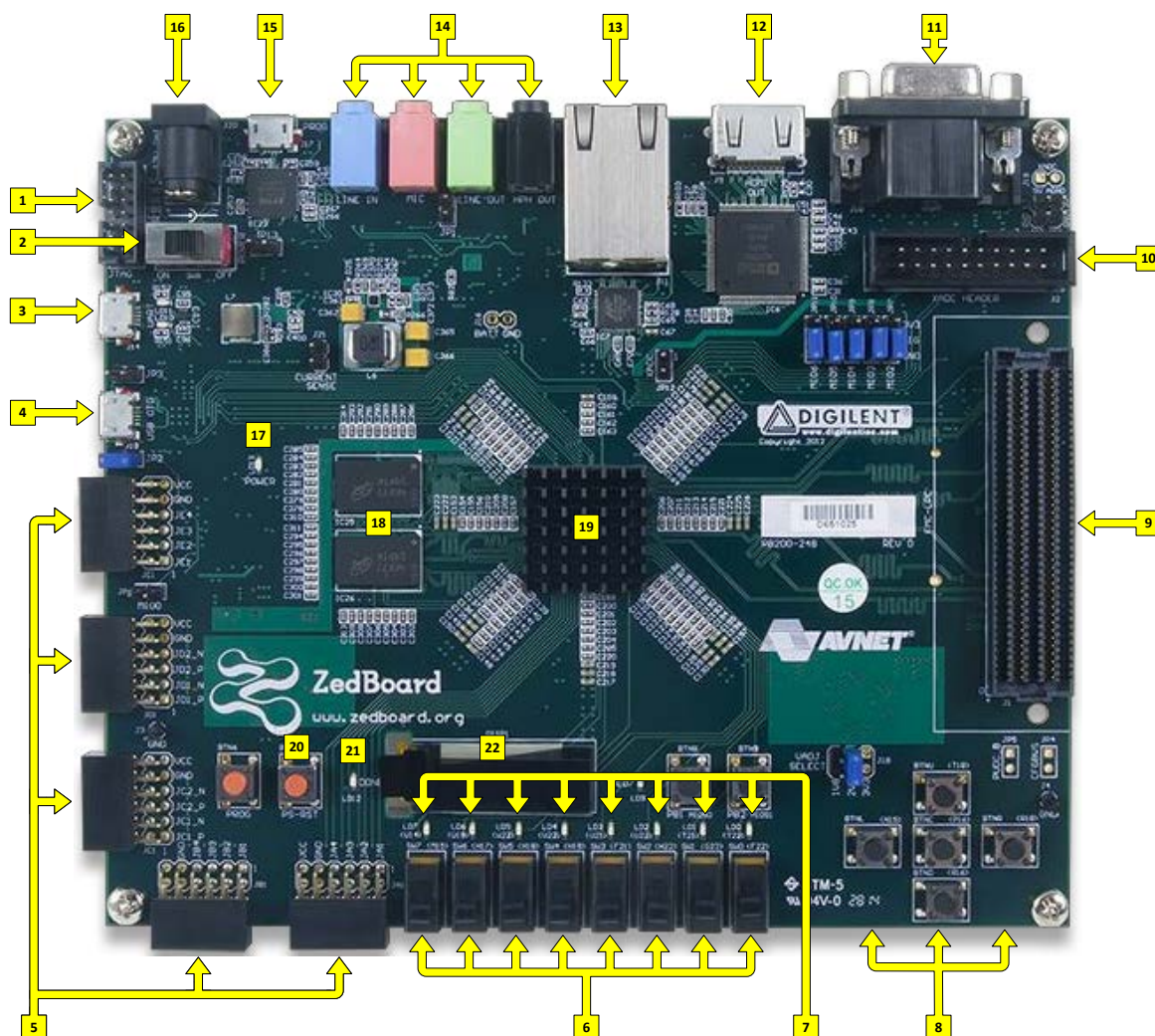


Figura 1. Vista superior de la tarjeta de desarrollo ZedBoard.

En la Tabla 1 se muestra la descripción de los diferentes elementos de la tarjeta de desarrollo ZedBoard destacados en la Figura 1.

Nº	Descripción	Nº	Descripción
1	Conector JTAG.	12	Conector HDMI.
2	Interruptor de encendido.	13	Conector Ethernet.
3	Conector UART.	14	Conectores de audio.
4	Conector USB OTG.	15	Conector USB.
5	Conectores Pmod.	16	Conector jack para alimentador externo.
6	Conmutadores deslizantes de usuario.	17	LED de alimentación.
7	LEDs de usuario.	18	Memorias DDR3 Micron.
8	Pulsadores de usuario.	19	FPFA ZINQ.
9	Conector FMC-LPC.	20	Pulsador de RESET.
10	Conector XADC.	21	LED DONE de programación de la FPGA.
11	Conector VGA.	22	Display OLED 128x32.

Tabla 1. Descripción de los elementos de la tarjeta de desarrollo ZedBoard.

2. Creación de un nuevo proyecto.

Se puede considerar un proyecto como un contenedor de todos los procesos y archivos necesarios para llevar a cabo un diseño, en nuestro caso basado en el lenguaje VHDL.

Cada proyecto realizado en Vivado se almacena en nuestro ordenador en una carpeta independiente, que contiene todos los archivos que el programa necesita para llevar a cabo las tareas mencionadas en el apartado anterior. Es decir, los ficheros fuente que escribamos, las simulaciones, los resultados intermedios y finales, los archivos de programación de la tarjeta de desarrollo a emplear, etc., se guardarán todos ellos en la carpeta correspondiente al proyecto, de manera que si copiamos dicha carpeta en otra ubicación, o en otro equipo, dispondremos de toda la información necesaria para la gestión del proyecto.

Antes de proceder a la creación de un proyecto ejecutaremos el software Vivado, bien desde el menú de aplicaciones de Windows, o bien haciendo click sobre un acceso directo colocado en el escritorio (Figura 2).

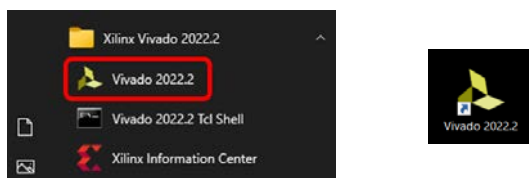


Figura 2. Opciones para ejecutar el software Vivado desde el sistema operativo Windows.

Tras la ejecución del software Vivado aparece una ventana como la mostrada en la Figura 3.

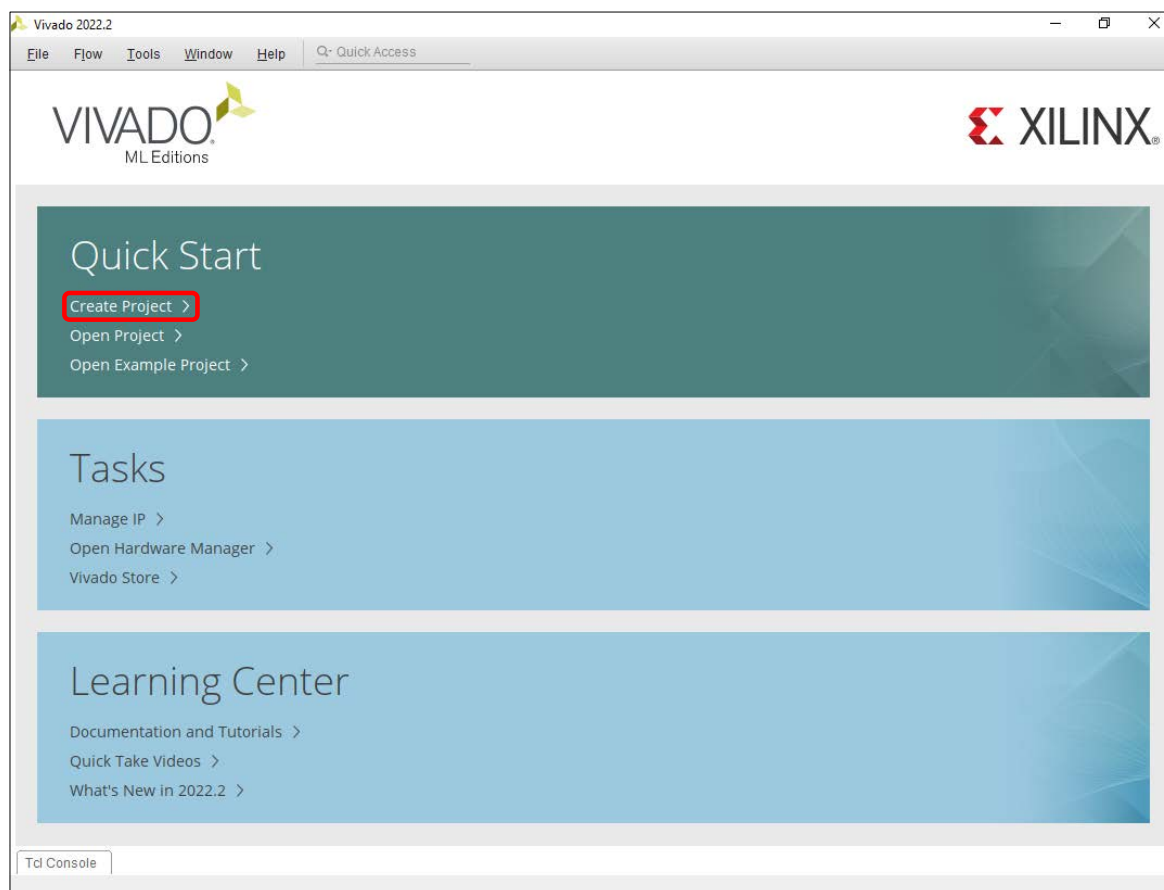


Figura 3. Ventana inicial del software Vivado.

En esta ventana, dentro de la sección **Quick Start**, haremos click sobre **Create Project**, con lo que el programa nos guiará mediante un asistente a través de las distintas tareas que debemos realizar para la creación de un nuevo proyecto.

Al hacer click sobre **Create Project** aparecerá la ventana de la Figura 4.

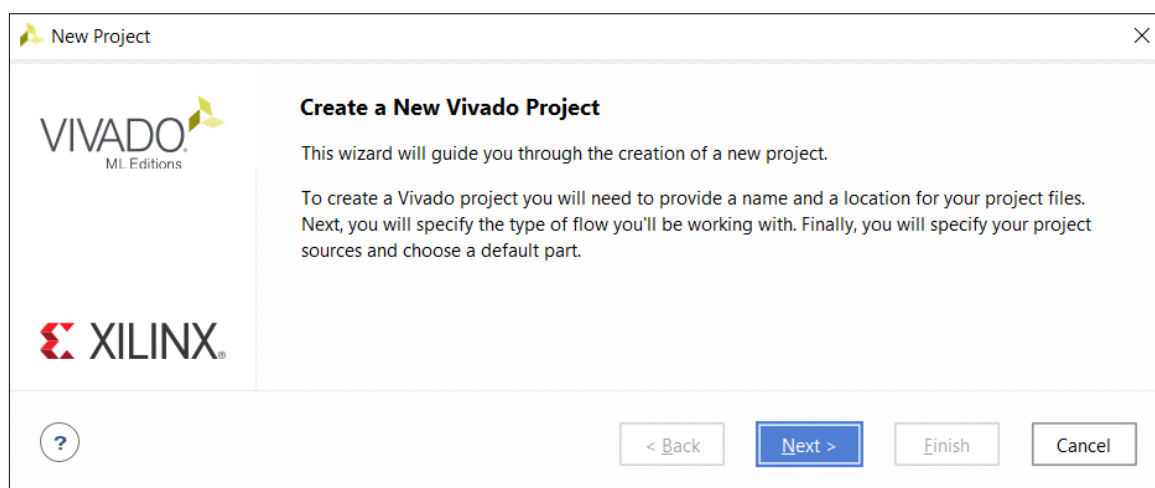


Figura 4. Ventana inicial de la opción Create Project.

Tras hacer click sobre el botón **Next** en la ventana anterior aparece la ventana de la Figura 5, donde introduciremos el nombre que deseamos asignar al proyecto y la ubicación donde

queremos que éste se almacene en nuestro ordenador. Deberemos dejar marcada la opción **Create project subdirectory** para que el programa cree la carpeta correspondiente.

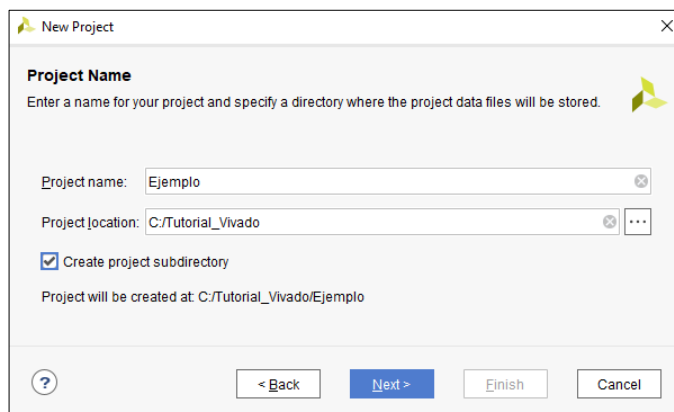


Figura 5. Indicación del nombre y la ubicación del proyecto a crear.

Tras hacer click sobre **Next** en la ventana anterior aparecerá la ventana de la Figura 6, donde seleccionaremos **RTL Project** como tipo de proyecto a crear y seguidamente haremos click sobre **Next**.

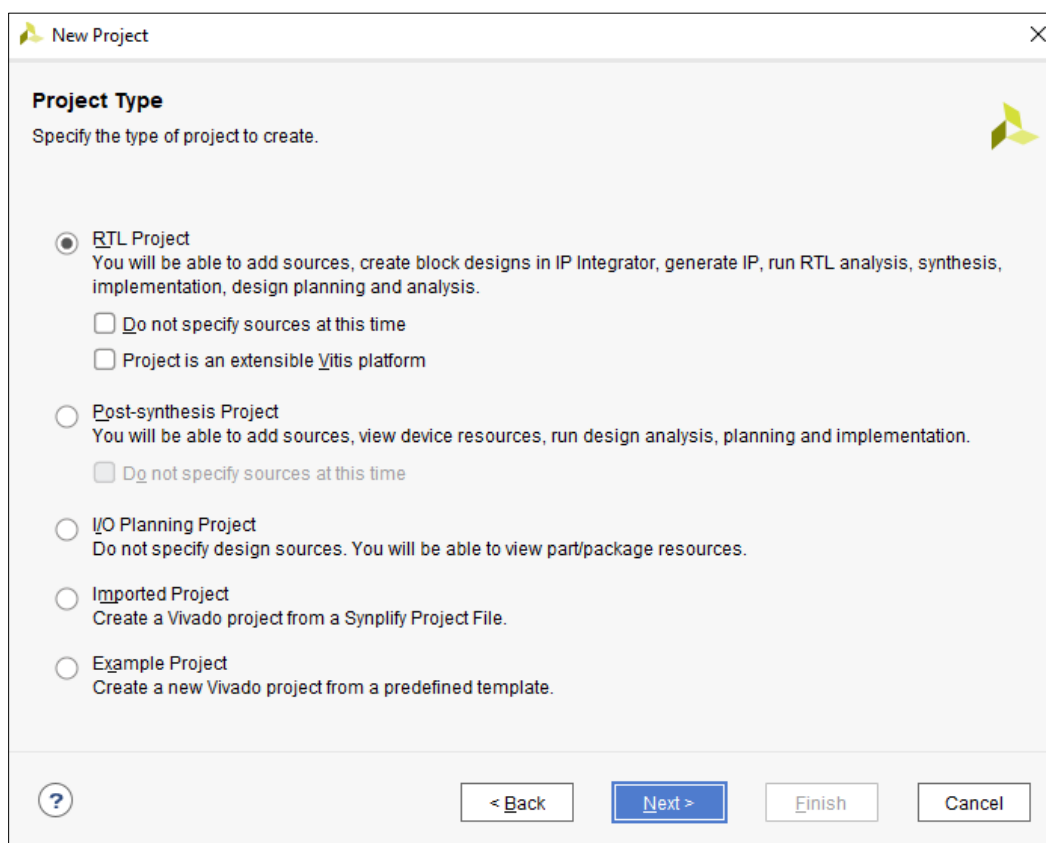


Figura 6. Elección del tipo de proyecto a crear.

En la siguiente ventana (Figura 7) podemos añadir a nuestro proyecto ficheros fuente ya existentes, por ejemplo, módulos VHDL que ya hayamos desarrollado, mediante la opción **Add Files**. También podemos crear nuevos archivos haciendo click sobre **Create File**.

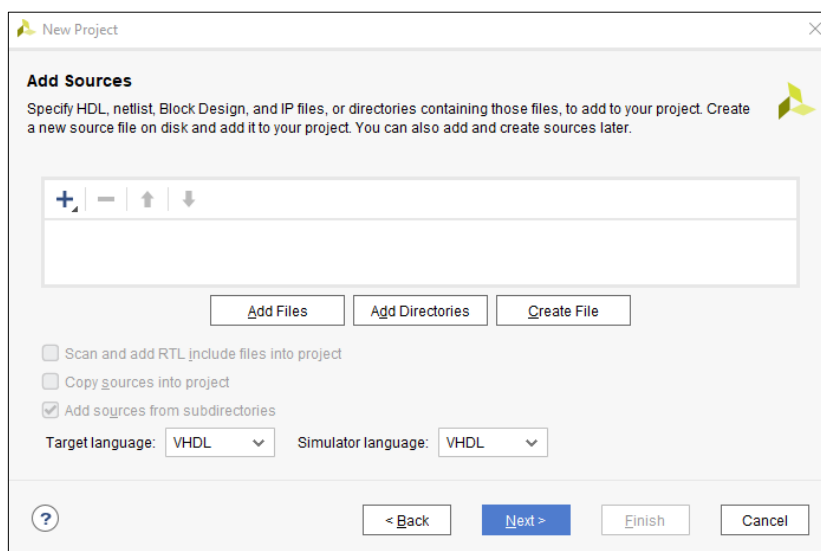


Figura 7. Adición al proyecto de ficheros fuente ya existentes o creación de nuevos ficheros.

En nuestro caso, haremos esto último, pero previamente, tanto en **Target language** como en **Simulator language**, seleccionaremos **VHDL** para asegurar el uso de este lenguaje en nuestro proyecto.

Para ilustrar de forma práctica el proceso de creación de un proyecto, vamos a desarrollar un ejemplo sencillo de tipo combinacional, consistente en el modelado de un sistema compuesto por las dos funciones lógicas de cuatro variables de entrada que se muestran a continuación:

$$F_1(D, C, B, A) = \overline{CBA} = \sum_4(0, 1, 2, 3, 4, 5, 6, 8, 9, 10, 11, 12, 13, 14)$$

$$F_2(D, C, B, A) = \overline{BA} + \overline{DC} = \sum_4(1, 5, 8, 9, 10, 11, 13)$$

Tras pulsar sobre **Create File**, aparecerá una ventana como la de la Figura 8, donde introduciremos el nombre que deseamos asignar al fichero fuente que contendrá el código VHDL necesario para la descripción de las funciones del ejemplo a desarrollar. En este caso, hemos introducido el texto **Circuito**, con lo que el fichero se guardará en la carpeta del proyecto con el nombre **Circuito.vhd**.

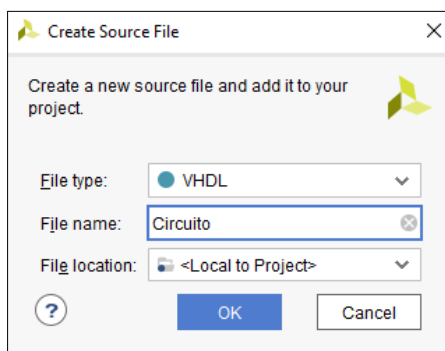


Figura 8. Asignación del nombre al fichero fuente a crear.

Una vez que hayamos introducido el nombre del fichero fuente y pulsado sobre **OK**, aparecerá la ventana de la Figura 9, donde se puede observar que ya aparece el fichero fuente a crear.

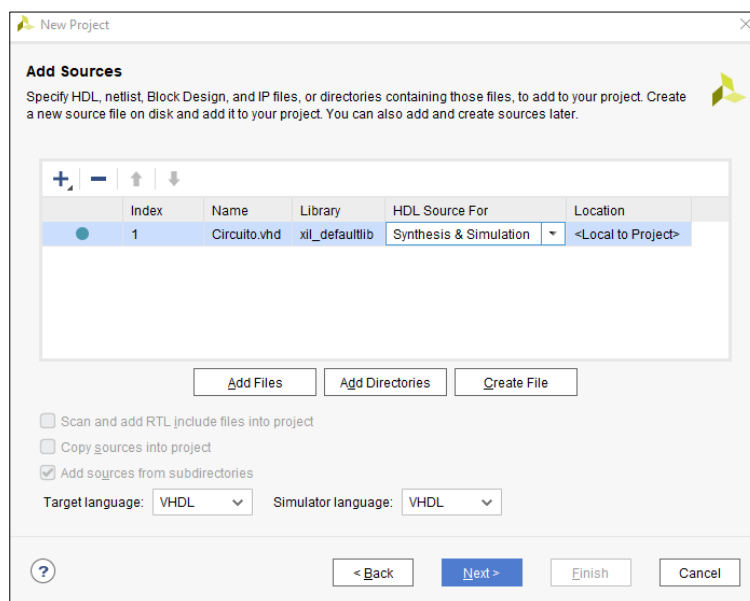


Figura 9. Presentación del fichero fuente a crear en la lista de ficheros fuente del proyecto.

En la opción **HDL Source For** del fichero fuente, seleccionaremos **Synthesis & Simulation**, ya que no sólo queremos simular nuestro diseño, sino sintetizarlo, es decir, generar un circuito cuyo funcionamiento se corresponda con el sistema descrito mediante el código VHDL en el fichero fuente.

Una vez hecho esto pulsaremos sobre **Next**, con lo que aparecerá la ventana de la Figura 10, que permite introducir las restricciones del proyecto.

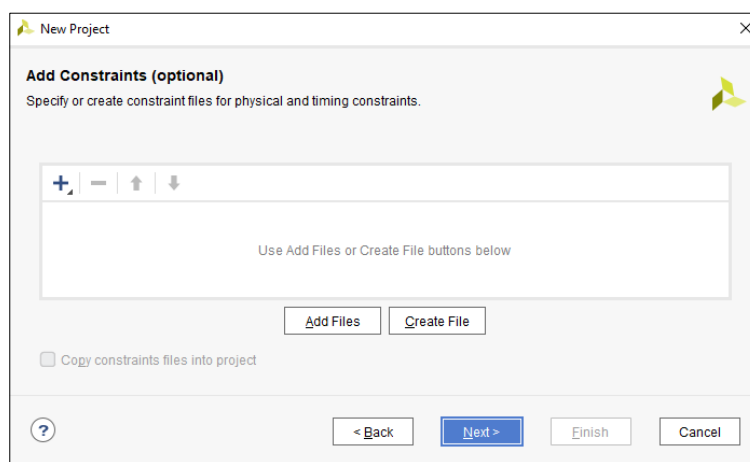


Figura 10. Ventana para la adición de ficheros de restricciones al proyecto.

Las restricciones son indicaciones al programa sobre cómo queremos implementar nuestro diseño en una determinada tarjeta de desarrollo. Por ejemplo, a qué terminales concretos de la FPGA conectaremos las entradas y salidas de nuestro circuito.

Por ahora no vamos a introducir restricciones al proyecto y simplemente pulsaremos sobre **Next**, con lo que aparecerá la ventana **Default Part**, que nos permitirá configurar las características de la tarjeta de desarrollo en la que se implementará el diseño. Para usar la tarjeta de desarrollo ZedBoard configuraremos esta ventana como se muestra en la Figura 11.

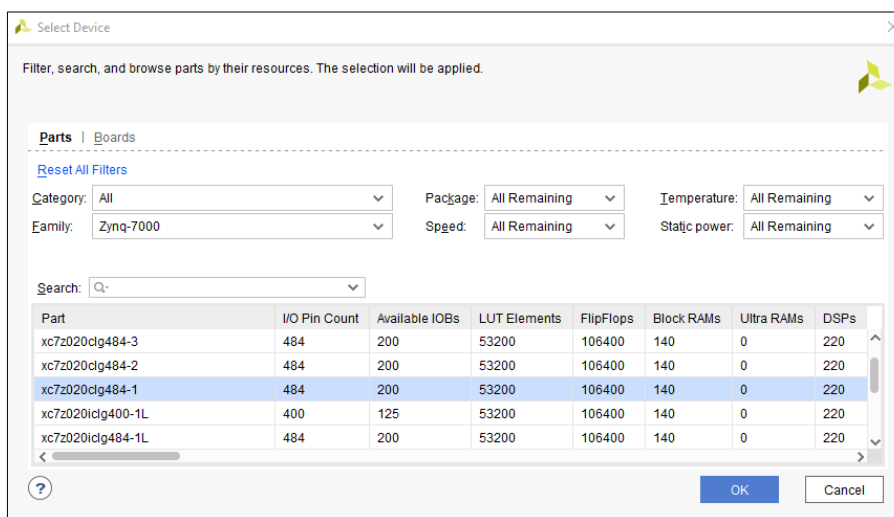


Figura 11. Configuración de la ventana **Default Part** para el empleo de la tarjeta de desarrollo ZedBoard.

Como se puede apreciar, se ha seleccionado en el campo **Family** la opción **Zynq-7000** y en el campo **Part** el circuito integrado **xc7z020clg484-1**, que es el chip que incorpora esta tarjeta.

Una vez que hayamos configurado las características de la tarjeta de desarrollo a emplear y pulsado sobre el botón **Next**, aparecerá una ventana como la de la Figura 12, donde se muestra un resumen de las características del proyecto especificadas hasta el momento, tales como el nombre asignado al mismo, el número de ficheros fuente añadidos, las propiedades de la tarjeta de desarrollo seleccionada, etc.

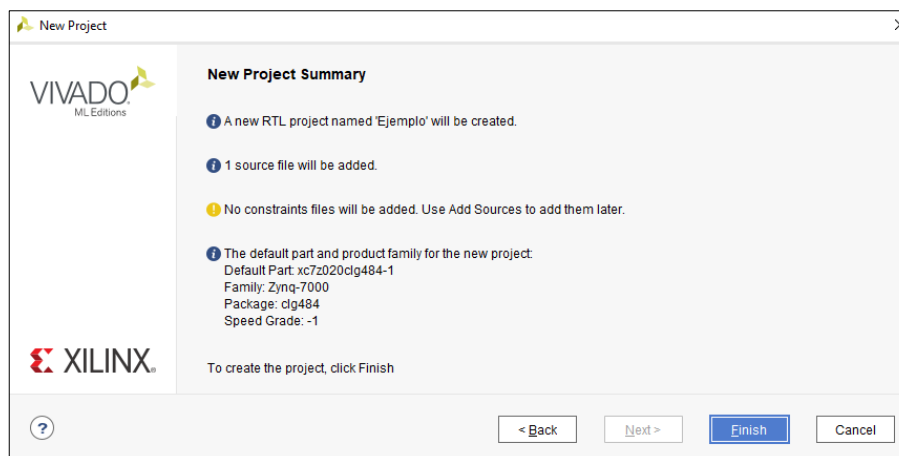


Figura 12. Resumen de las características del proyecto creado.

Tras hacer click sobre **Finish**, Vivado empieza a generar el proyecto y nos lo indica mediante una ventana como la que se muestra en la Figura 13.

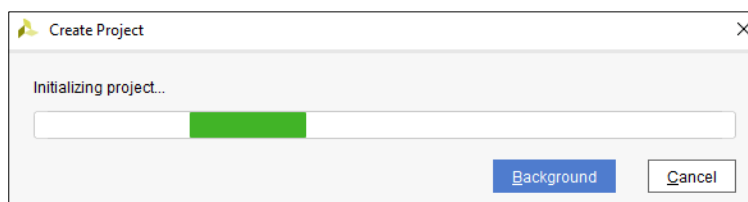


Figura 13. Inicialización de la creación del proyecto en Vivado.

Seguidamente, el software muestra una nueva ventana (**Define Module**) donde nos pregunta sobre datos concretos del fichero fuente definido anteriormente (**Circuito.vhd**). En ella, podemos asignar nombres a la entidad y a la arquitectura (o mantener los que el programa nos propone por defecto), así como especificar las características de los terminales de entrada y de salida del sistema. En nuestro caso, el circuito posee cuatro variables de entrada (D, C, B y A) y dos variables de salida (F1 y F2), por lo que hemos configurado esta ventana tal como se muestra en la Figura 14.

Define a module and specify I/O Ports to add to your source file.
For each port specified:
MSB and LSB values will be ignored unless its Bus column is checked.
Ports with blank names will not be written.

Module Definition

Entity name: CIRCUITO

Architecture name: A_CIRCUITO

I/O Port Definitions

Port Name	Direction	Bus	MSB	LSB
D	in	☐	0	0
C	in	☐	0	0
B	in	☐	0	0
A	in	☐	0	0
F1	out	☐	0	0
F2	out	☐	0	0

OK Cancel

Figura 14. Configuración de las características generales del módulo fuente VHDL.

En el caso de que algunas entradas o salidas del circuito no fueran señales simples, sino combinaciones de varios bits, seleccionaríamos la opción **Bus**, e introduciríamos en MSB el subíndice del bit de mayor peso y en LSB el del bit de menor peso. Por ejemplo, si asignamos a MSB el valor 4 y a LSB el valor 0, se creará un vector de entrada de tipo **std_logic_vector (4 downto 0)**.

Tras especificar las características del módulo fuente en la ventana anterior y hacer click sobre **OK** aparecerá la ventana principal del entorno Vivado (Figura 15), que muestra las diferentes secciones del proyecto creado.

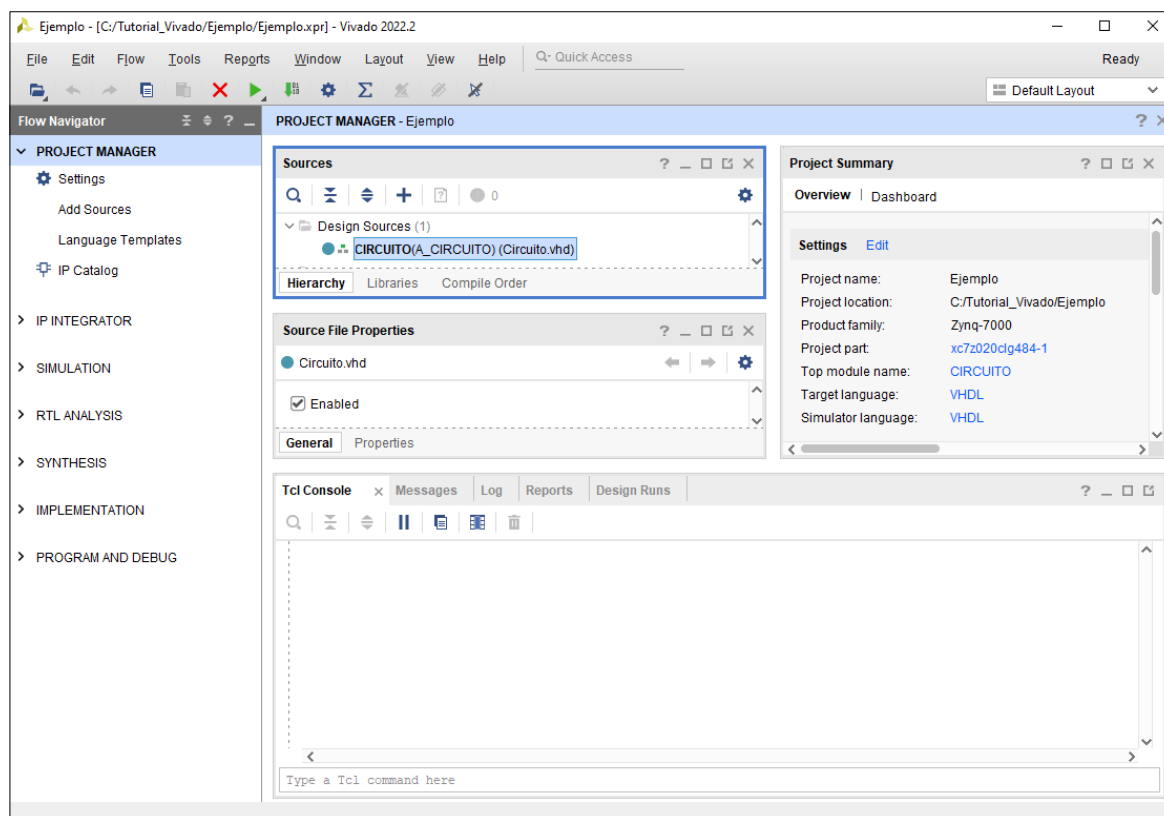


Figura 15. Pantalla principal del software Vivado correspondiente al proyecto.

En esta ventana podemos observar como en la sección **Sources** aparece el fichero fuente definido anteriormente (**Circuito.vhd**), así como el nombre de su entidad (**Circuito**) y el de su arquitectura (**A_Circuito**).

3. Edición del fichero fuente.

Una vez creado el proyecto, el siguiente paso será completar el contenido del fichero fuente VHDL. Para ello, haremos doble clic sobre dicho fichero fuente en la sección **Sources** de la pantalla principal de Vivado, con lo cual, el programa abrirá este fichero en una nueva ventana ubicada en la parte derecha de la pantalla, donde podremos editarlo para añadir el contenido necesario, modificarlo, etc. En la Figura 16 se muestra el contenido del fichero fuente de nuestro ejemplo una vez editado para incluir las ecuaciones que definen las funciones de salida F1 y F2.

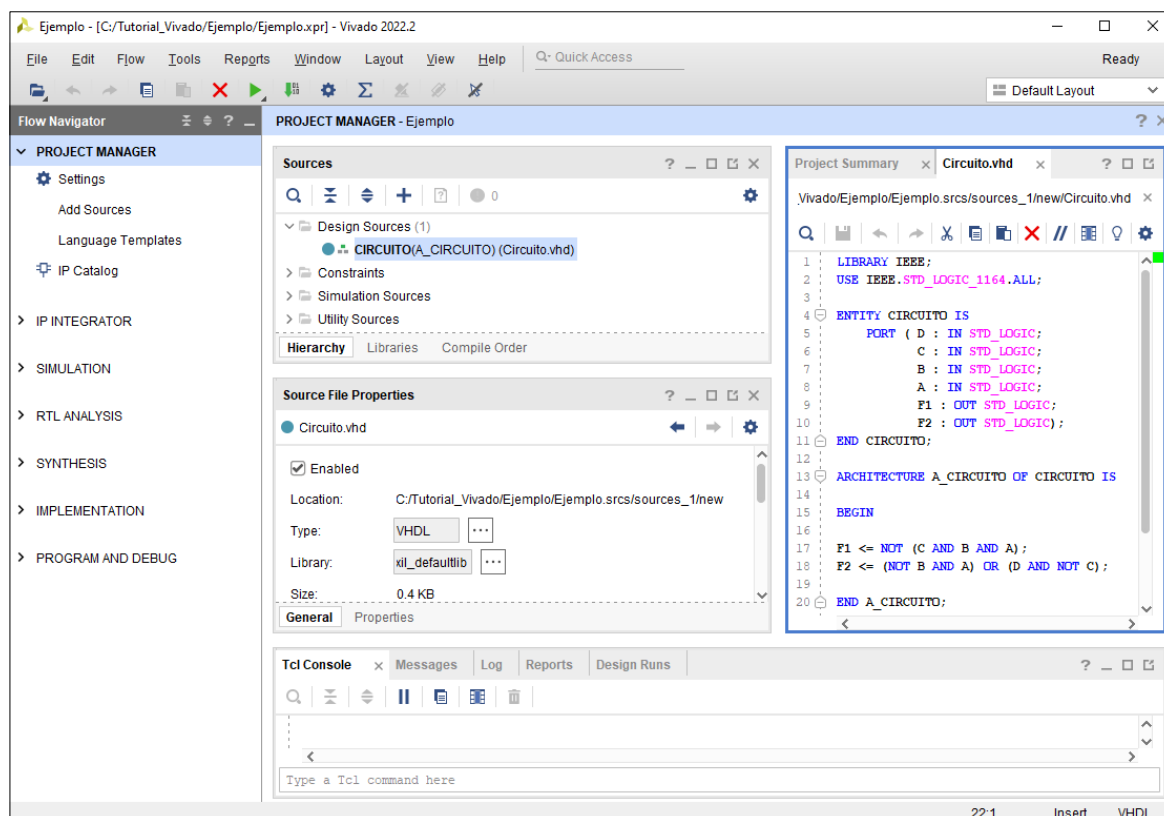


Figura 16. Contenido del fichero fuente VHDL editado para incluir las ecuaciones del ejemplo.

4. Simulación del sistema.

Una vez editado y completado el fichero fuente, ya podremos llevar a cabo la simulación del diseño para comprobar que su comportamiento se corresponde con las especificaciones.

Para poder simular el diseño, previamente deberemos añadir un nuevo fichero fuente al proyecto, que en este caso será del tipo **VHDL Test Bench**. Para ello, en la zona **Flow Navigator** de la pantalla principal de Vivado, desplegaremos **Project Manager** y seleccionaremos **Add Sources**, o bien, haremos click sobre el signo + en la sección **Sources**.

Una vez hecho esto, aparecerá una ventana emergente donde seleccionaremos la opción **Add o create simulation sources**, como puede apreciarse en la Figura 17.

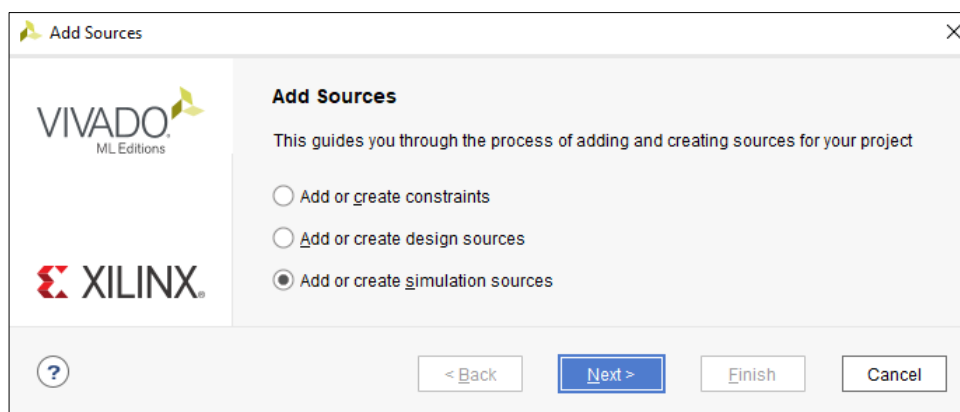


Figura 17. Selección del tipo simulación (VHDL Test Bench) para el nuevo fichero fuente.

Una vez seleccionada la opción **Add o create simulation sources** para el nuevo fichero fuente y validada mediante la activación de **Next**, aparecerá la ventana de la Figura 18, mediante la cual podremos añadir a nuestro proyecto ficheros fuente de simulación ya existentes, activando la opción **Add Files**, o crear ficheros fuente nuevos haciendo click sobre **Create File**.

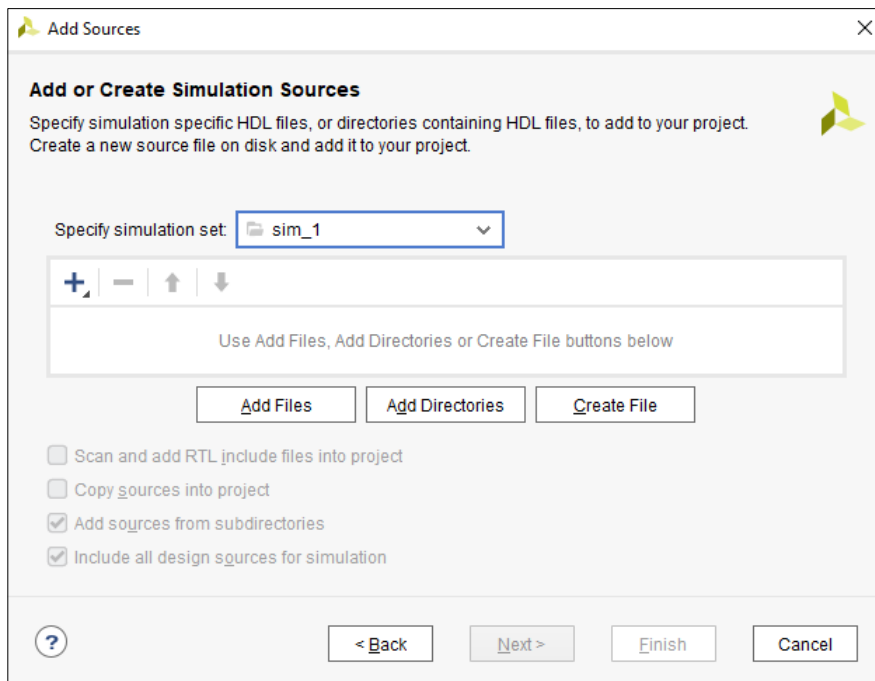


Figura 18. Adición al proyecto de ficheros fuente de simulación o creación de nuevos ficheros.

En nuestro caso, haremos esto último, con lo cual aparecerá una ventana como la de la Figura 19, en la que asignaremos un nombre al nuevo fichero de simulación a crear (**T_Circuito**).

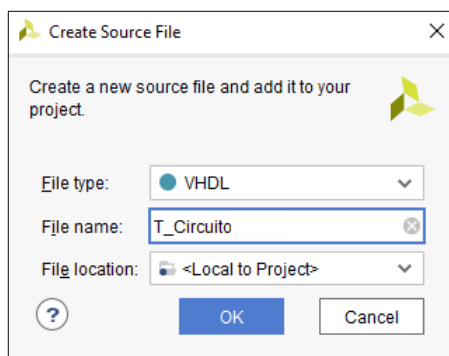


Figura 19. Asignación del nombre al nuevo fichero fuente de simulación.

Una vez que hayamos introducido el nombre del nuevo fichero de simulación y pulsado sobre **OK**, aparecerá una ventana como la de la Figura 20, donde se puede observar que ya aparece el fichero de simulación a crear.

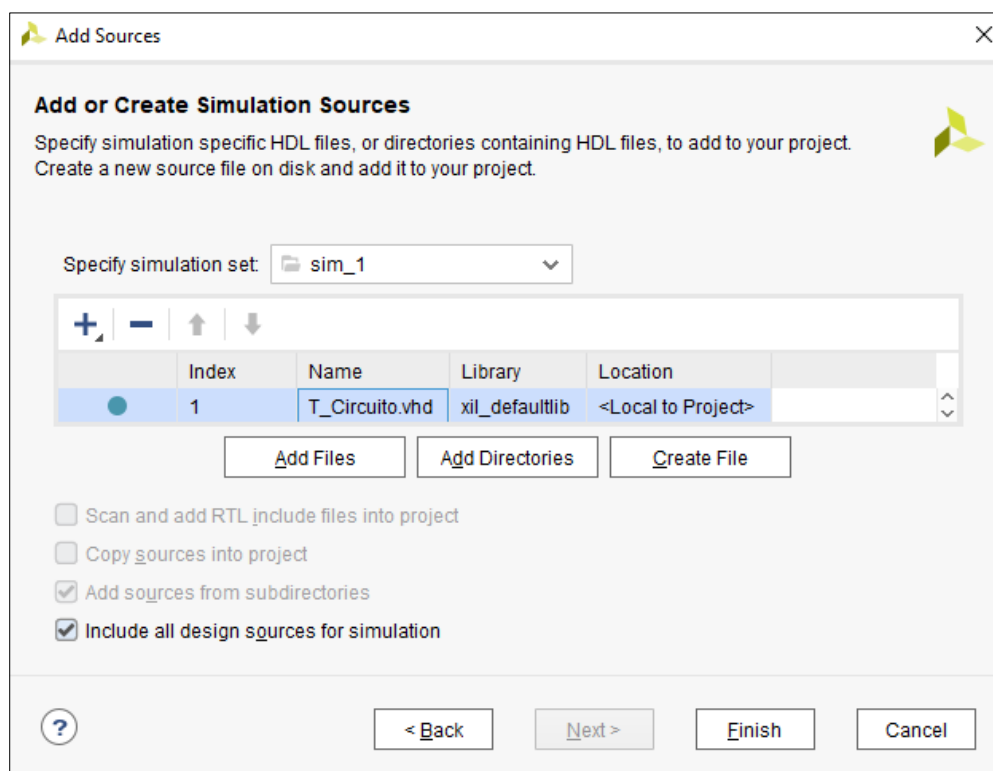


Figura 20. Presentación del fichero de simulación a crear en la lista de ficheros fuente del proyecto .

Tras pulsar sobre **Finish** se mostrará una nueva ventana en la que podremos asignar nombres tanto a la entidad como a la arquitectura del fichero de simulación. En nuestro caso la configuraremos como se muestra en la Figura 21 y a continuación haremos click sobre **OK**.

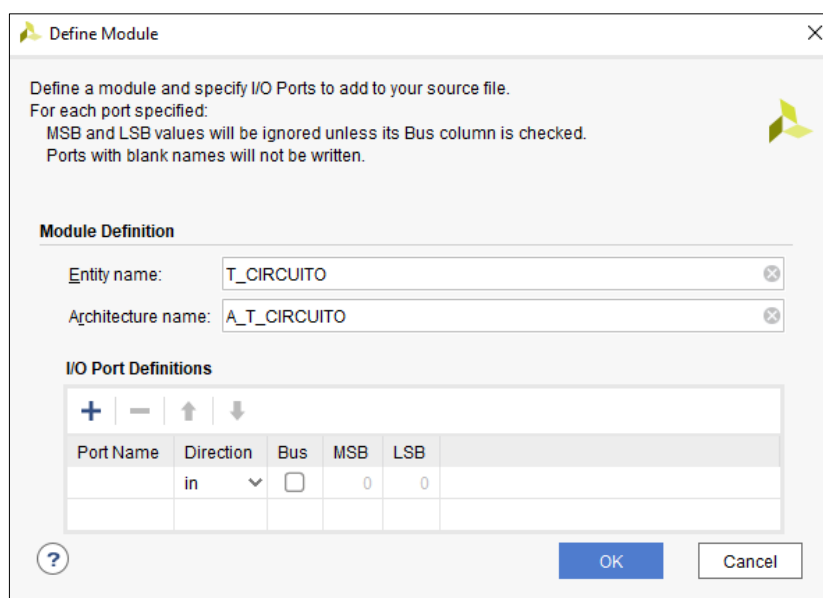


Figura 21. Configuración de las características del fichero de simulación.

Una vez validados los datos del fichero de simulación introducidos mediante **OK**, se generará una versión preliminar del mismo, que se mostrará en la sección **Sources** de la pantalla principal de Vivado dentro de una carpeta denominada **Simulation Sources**, como se puede apreciar en la Figura 22.

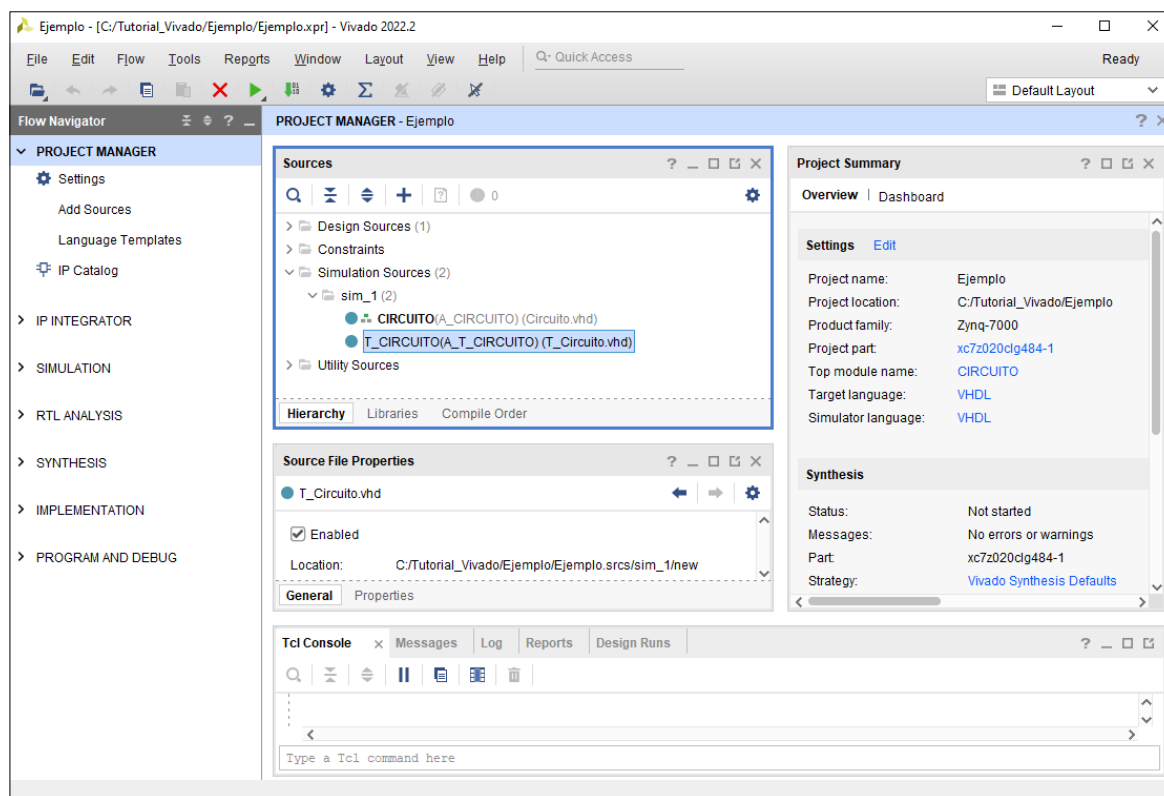


Figura 22. Localización del fichero de simulación en la ventana principal de Vivado.

Para editar el fichero de simulación creado previamente, haremos doble click sobre el nombre de éste (**T_Circuito.vhd**), con lo cual, el programa abrirá el fichero en una nueva ventana ubicada en la parte derecha de la pantalla, donde podremos editarlo para añadir el contenido necesario, modificarlo, etc., como se muestra en la Figura 23.

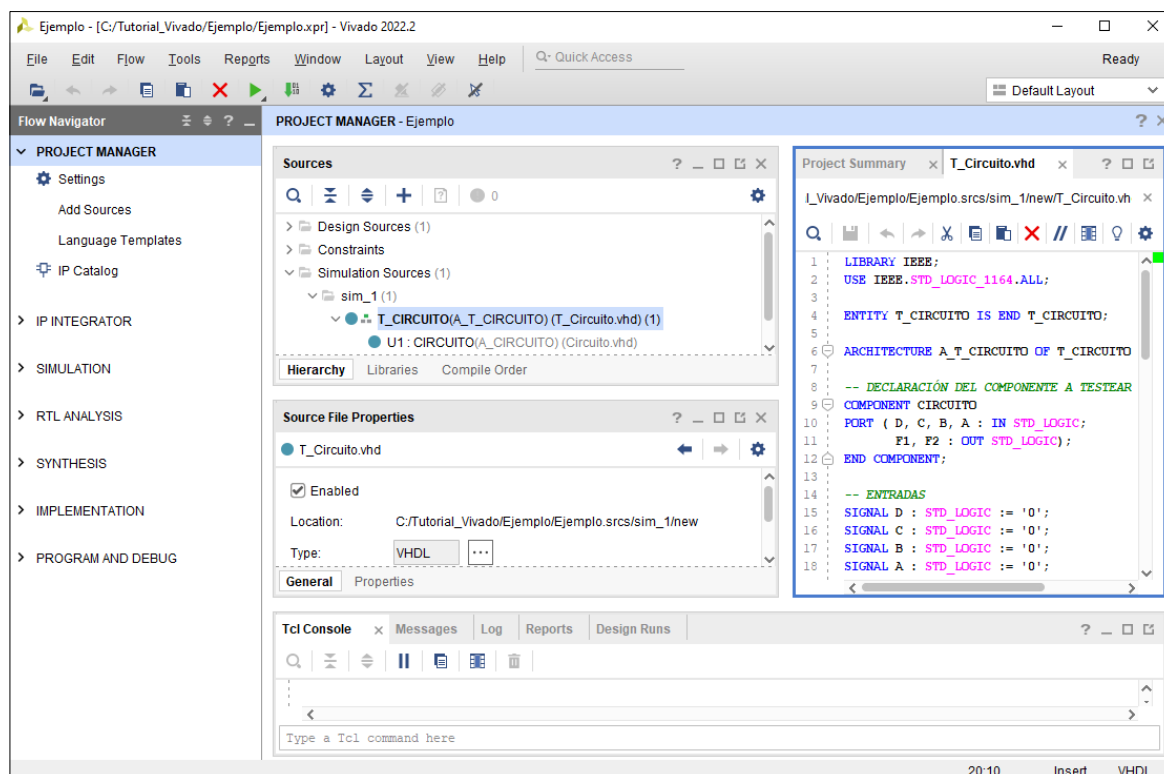


Figura 23. Ventana de edición del fichero de simulación.

La forma de editar el fichero de simulación de manera adecuada se explica en otro tutorial, que también se encuentra disponible en la sección de prácticas de la plataforma moodle de la asignatura Fundamentos de Computadores.

Una vez editado el fichero de simulación, el código completo del mismo es el que se muestra a continuación:

```

LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;

ENTITY T_CIRCUITO IS END T_CIRCUITO;

ARCHITECTURE A_T_CIRCUITO OF T_CIRCUITO IS

-- DECLARACIÓN DEL COMPONENTE A TESTEAR (UUT)
COMPONENT CIRCUITO
    PORT (D, C, B, A : IN STD_LOGIC;
          F1, F2 : OUT STD_LOGIC);
END COMPONENT;

-- ENTRADAS
SIGNAL D : STD_LOGIC := '0';
SIGNAL C : STD_LOGIC := '0';
SIGNAL B : STD_LOGIC := '0';
SIGNAL A : STD_LOGIC := '0';

--SALIDAS
SIGNAL F1 : STD_LOGIC;
SIGNAL F2 : STD_LOGIC;

BEGIN

-- INSTANCIACIÓN DEL COMPONENTE (UUT)
U1: CIRCUITO PORT MAP ( A => A,
                        B => B,
                        C => C,
                        D => D,
                        F1 => F1,
                        F2 => F2);

-- DEFINICIÓN DE ESTÍMULOS
STIM_PROC: PROCESS
BEGIN
    D <= '0'; C <= '0'; B <= '0'; A <= '0'; WAIT FOR 10 NS;
    D <= '0'; C <= '0'; B <= '0'; A <= '1'; WAIT FOR 10 NS;
    D <= '0'; C <= '0'; B <= '1'; A <= '0'; WAIT FOR 10 NS;
    D <= '0'; C <= '0'; B <= '1'; A <= '1'; WAIT FOR 10 NS;
    D <= '0'; C <= '1'; B <= '0'; A <= '0'; WAIT FOR 10 NS;
    D <= '0'; C <= '1'; B <= '0'; A <= '1'; WAIT FOR 10 NS;
    D <= '0'; C <= '1'; B <= '1'; A <= '0'; WAIT FOR 10 NS;
    D <= '0'; C <= '1'; B <= '1'; A <= '1'; WAIT FOR 10 NS;
    D <= '1'; C <= '0'; B <= '0'; A <= '0'; WAIT FOR 10 NS;
    D <= '1'; C <= '0'; B <= '0'; A <= '1'; WAIT FOR 10 NS;
    D <= '1'; C <= '0'; B <= '1'; A <= '0'; WAIT FOR 10 NS;
    D <= '1'; C <= '0'; B <= '1'; A <= '1'; WAIT FOR 10 NS;
    D <= '1'; C <= '1'; B <= '0'; A <= '0'; WAIT FOR 10 NS;
    D <= '1'; C <= '1'; B <= '0'; A <= '1'; WAIT FOR 10 NS;
    D <= '1'; C <= '1'; B <= '1'; A <= '0'; WAIT FOR 10 NS;
    D <= '1'; C <= '1'; B <= '1'; A <= '1'; WAIT FOR 10 NS;

```

```

D <= '1'; C <= '1'; B <= '1'; A <= '0'; WAIT FOR 10 NS;
D <= '1'; C <= '1'; B <= '1'; A <= '1'; WAIT;
END PROCESS;

END A_T_CIRCUITO;

```

A continuación, pasaremos a ejecutar la simulación del circuito para verificar su correcto comportamiento. Para ello, en la zona **Flow Navigator** situada a la izquierda de la pantalla principal de Vivado, desplegaremos la opción **Simulation**, seleccionaremos **Run Simulation** y ejecutaremos **Run behavioral Simulation** para realizar una simulación de comportamiento (Figura 24). Si antes de hacer esto no hemos salvado los ficheros, Vivado nos preguntará si deseamos hacerlo.

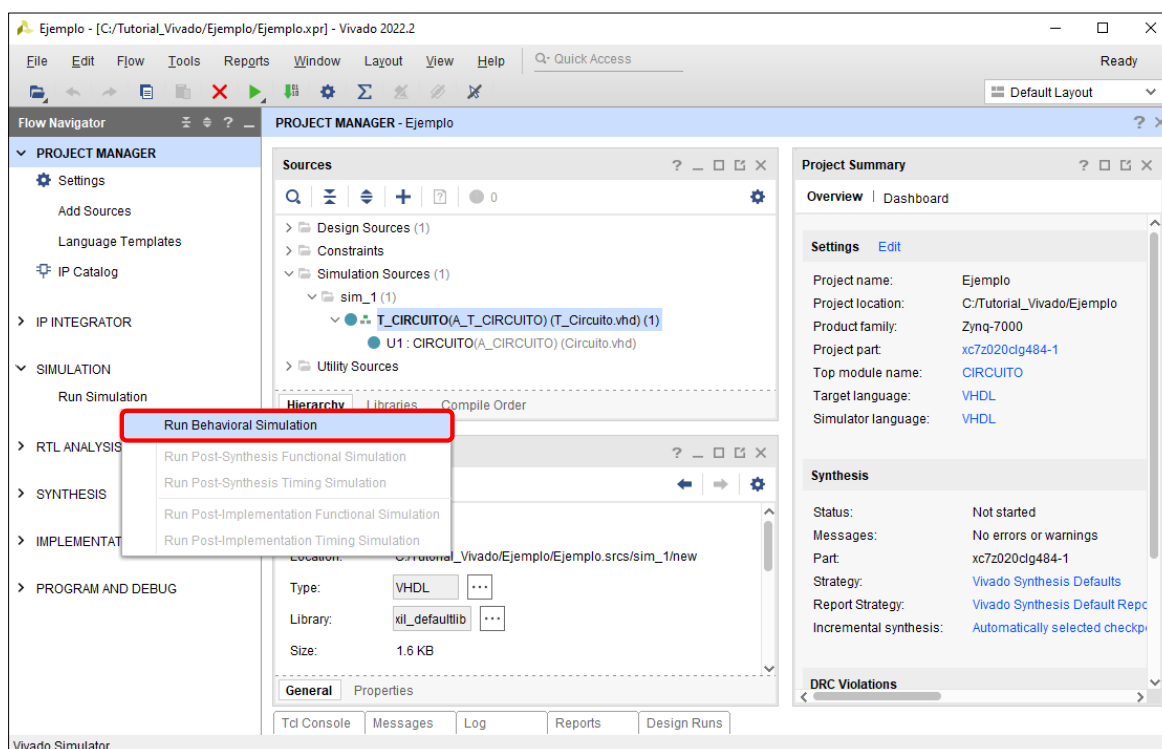


Figura 24. Ejecución de la simulación.

Una vez hecho esto comienza la ejecución de la simulación, lo cual nos indica el programa mediante una ventana como la que se muestra en la Figura 25.

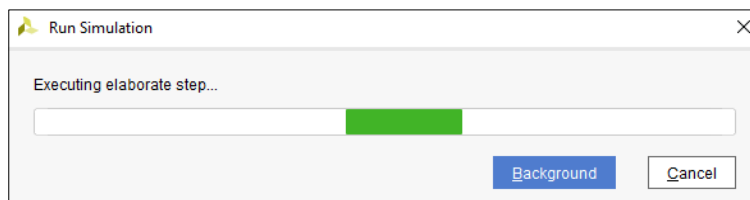


Figura 25. Inicio de la simulación del circuito en Vivado.

Una vez finalizada la simulación, Vivado mostrará el resultado de la misma en forma de cronograma, donde podremos observar los valores que hemos asignado a las variables de entrada D, C, B y A (vectores de entrada de test), así como los resultados obtenidos en la simulación para las salidas F1 y F2.

Para facilitar el análisis del cronograma, podremos cambiar los colores de las señales, ajustar el tiempo de simulación, ampliar o reducir el intervalo de tiempo representado en la pantalla, etc., como se muestra en la Figura 26.

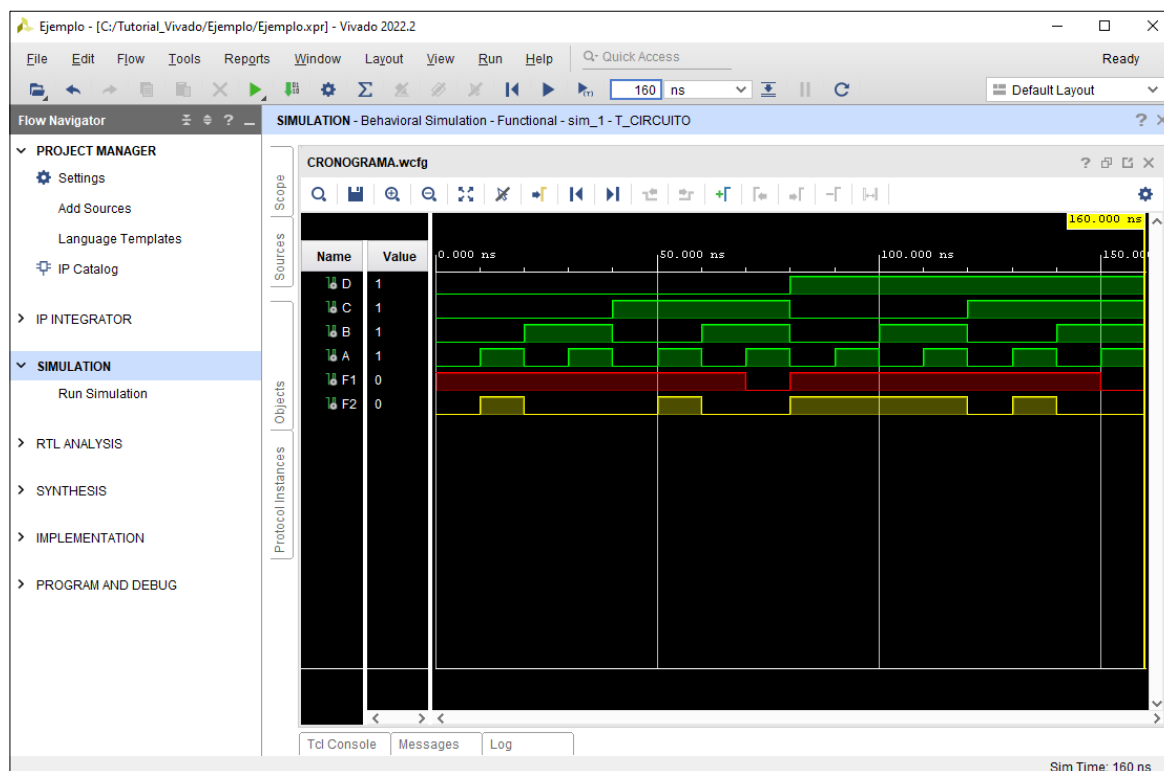


Figura 26. Cronograma resultante de la simulación del circuito del ejemplo.

Por último, observaremos detenidamente el cronograma y comprobaremos que los valores de las salidas son los esperados para cada una de las combinaciones de las variables de entrada.

En caso de que los resultados observados en el cronograma no se correspondan con los esperados, deberemos editar nuevamente el código VHDL y volver a simular el sistema reiteradamente hasta conseguir subsanar todas las anomalías de funcionamiento detectadas.

5. Síntesis del sistema.

Una vez llevada a cabo la simulación del diseño y comprobado que éste se comporta de manera satisfactoria, procederemos a realizar la síntesis del diseño, mediante la cual el programa generará un circuito físico que cumpla con los requisitos especificados en el código VHDL del fichero fuente.

Para ello, en la zona **Flow Navigator**, situada en la parte izquierda de la pantalla principal de Vivado, desplegaremos la opción **Synthesis** y a continuación ejecutaremos **Run Synthesis** (Figura 27).

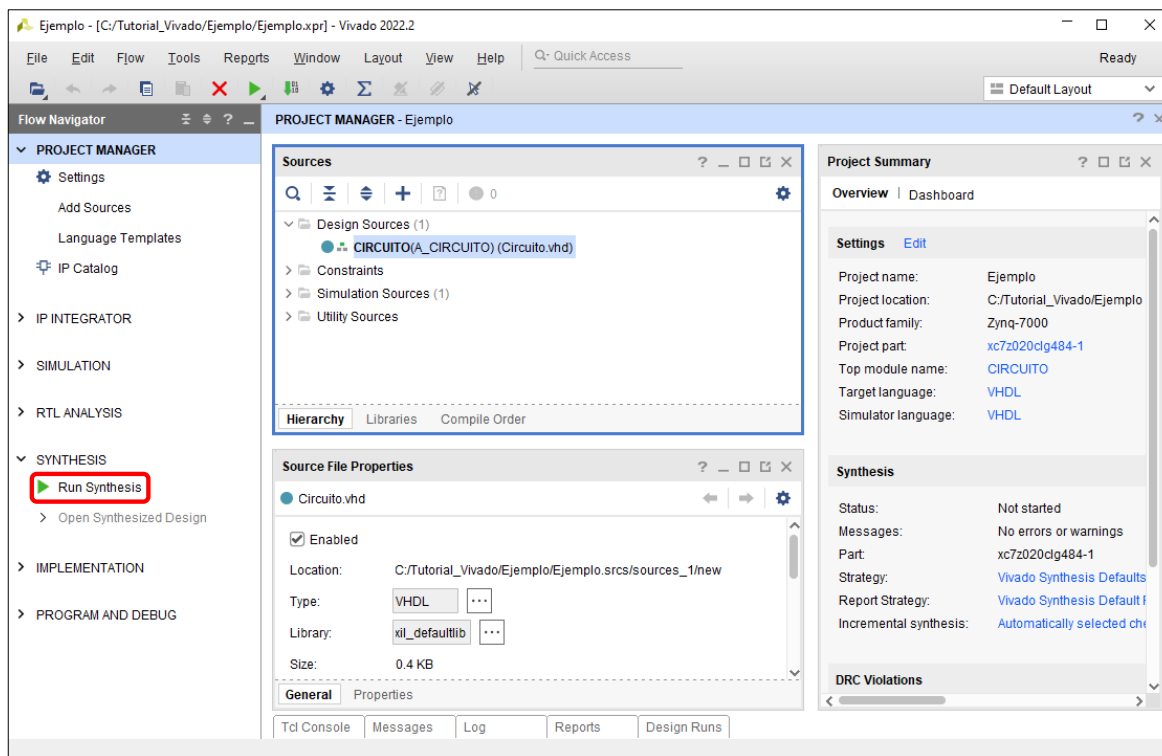


Figura 27. Ejecución de la síntesis del diseño.

Tras ejecutar la opción Run Synthesis aparecerá la ventana de la figura 21, donde se nos solicitará la confirmación antes de iniciar la síntesis del diseño.

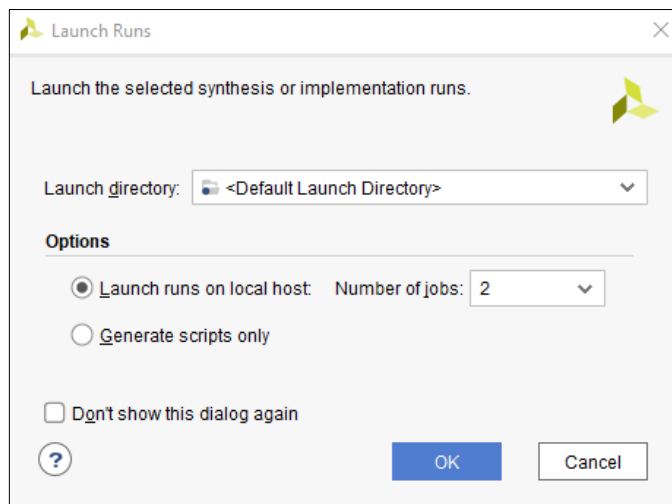


Figura 28. Confirmación de la ejecución de la síntesis del diseño.

Una vez realizada la confirmación mediante la activación de botón **OK**, en la parte superior derecha de la pantalla principal de VIVADO podremos observar la evolución del proceso de síntesis, como se aprecia en la Figura 29.

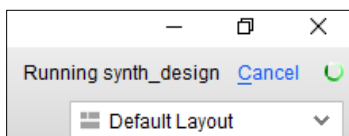


Figura 29. Síntesis del diseño en proceso de ejecución.

Si la síntesis del diseño concluye sin errores, el programa nos lo indicará mediante una ventana emergente como la mostrada en la Figura 30, que además nos permitirá iniciar el proceso de implementación del diseño, abrir el diseño sintetizado o visualizar los informes relativos al proceso realizado. Seleccionaremos **View Reports** y haremos click sobre **OK**.

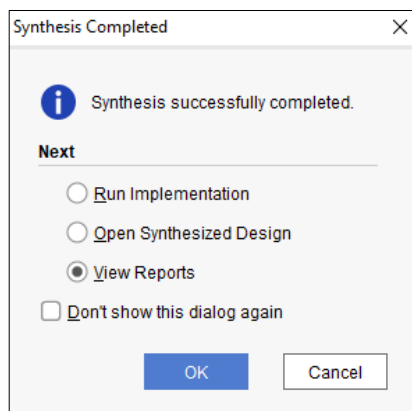


Figura 30. Comunicación de la finalización satisfactoria del proceso de síntesis.

6. Implementación del sistema.

La siguiente fase del proceso de diseño es la implementación del sistema. En esta fase se lleva a cabo la transformación del circuito modelado previamente para permitir su posterior implementación en un dispositivo FPGA integrado en una tarjeta de desarrollo.

Para poder indicar a Vivado cuales son los terminales de la FPGA a los que deseamos conectar las entradas y salidas de nuestro circuito (D, C, B, A, F1 y F2), antes de llevar a cabo la implementación propiamente dicha, deberemos añadir un nuevo fichero al proyecto. Este fichero recibe el nombre de fichero de restricciones y contiene información acerca de todos los elementos de entrada y de salida de la tarjeta que podemos usar.

En nuestro caso, vamos a conectar las entradas D, C, B y A de nuestro circuito a cuatro interruptores para poder cambiar sus valores de forma interactiva y las salidas F1 y F2 las vamos a dirigir a dos de los LEDs de la tarjeta de desarrollo empleada.

Es importante comprender que cada tarjeta de desarrollo tiene una estructura distinta e incorpora una FPGA diferente, por lo que cada modelo concreto de tarjeta posee su propio fichero de restricciones. En Vivado se emplean ficheros de restricciones con formato XDC, que es un formato de texto que podemos editar dentro del entorno este software para configurar las entradas y salidas de nuestro circuito.

Para implementar el sistema en la tarjeta ZedBoard, deberemos añadir al proyecto el fichero de restricciones correspondiente a dicha tarjeta. Este fichero es proporcionado por el propio fabricante de la tarjeta y su nombre es **ZedBoard-Master.xdc**. También se encuentra disponible en la sección de prácticas de la plataforma moodle de la asignatura Fundamentos de Computadores.

Para añadir al proyecto el fichero de restricciones, seleccionaremos **Add Sources** en la pantalla principal de Vivado y a continuación, en la ventana que aparecerá, ejecutaremos la opción **Add or create constraints**, como se muestra en la Figura 31.

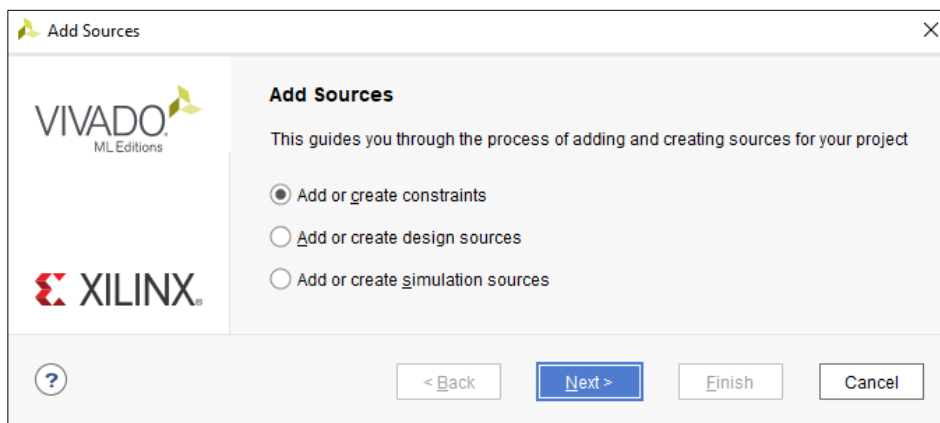


Figura 31. Selección del tipo fichero de restricciones para el nuevo fichero fuente.

Tras pulsar sobre **Next** aparecerá la ventana de la Figura 32, en la que haremos click sobre **Add Files**, ya que disponemos del fichero de restricciones de la tarjeta proporcionado por el fabricante.

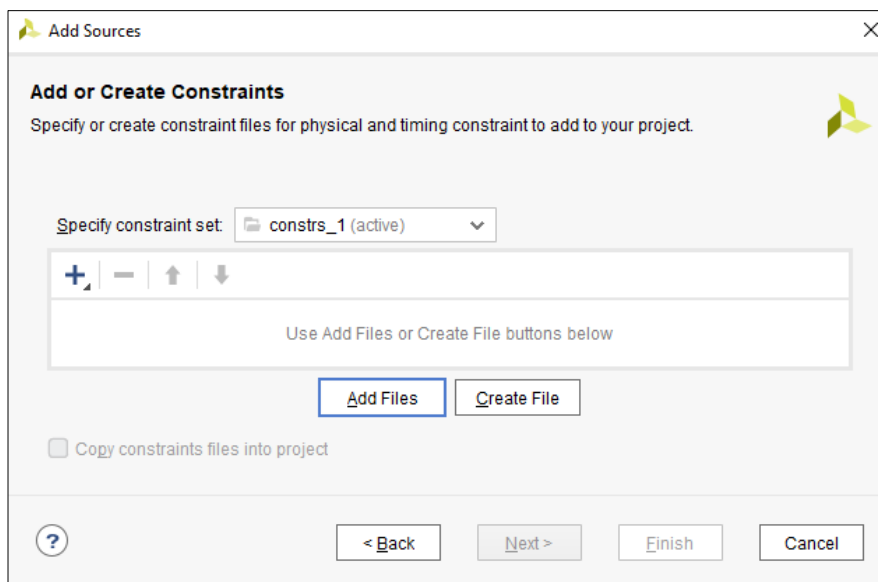


Figura 32. Adición al proyecto de un fichero de restricciones o creación de uno nuevo.

A continuación, seleccionaremos el fichero de restricciones **Zedboard-Master.xdc** en la ubicación donde lo hayamos guardado previamente y pulsaremos sobre **OK**, con lo cual aparecerá la ventana de la Figura 33, donde se puede observar que ya se muestra el fichero de restricciones añadido.

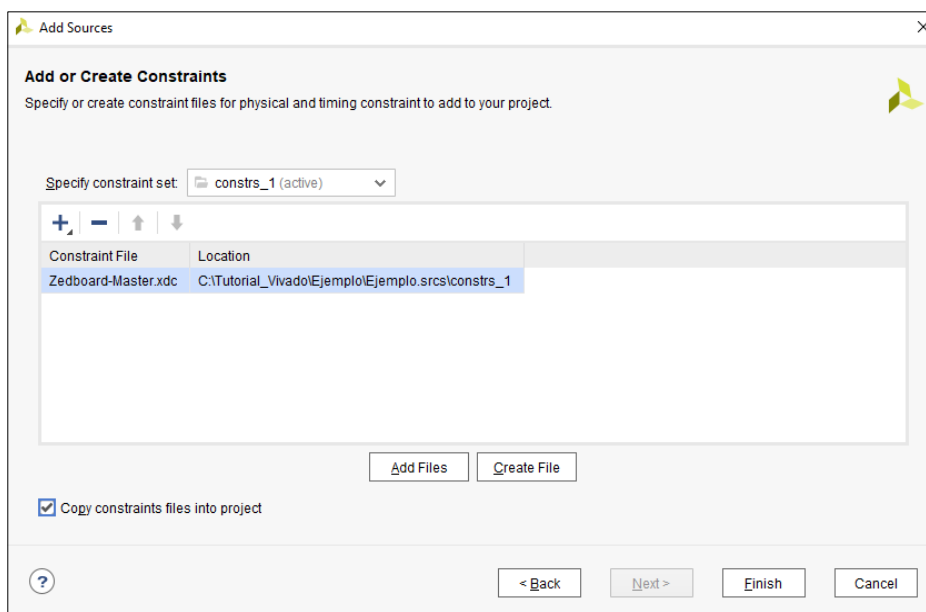


Figura 33. Presentación del fichero de restricciones de la tarjeta ZedBoard en la lista de ficheros del proyecto.

En la figura anterior se puede apreciar que hemos seleccionado la opción **Copy constraints files into project**, lo cual provocará que Vivado asocie el fichero de restricciones al proyecto.

Tras pulsar sobre **Finish**, observaremos que en la ventana principal de Vivado ya aparece asociado al proyecto el fichero de restricciones añadido (Figura 34).

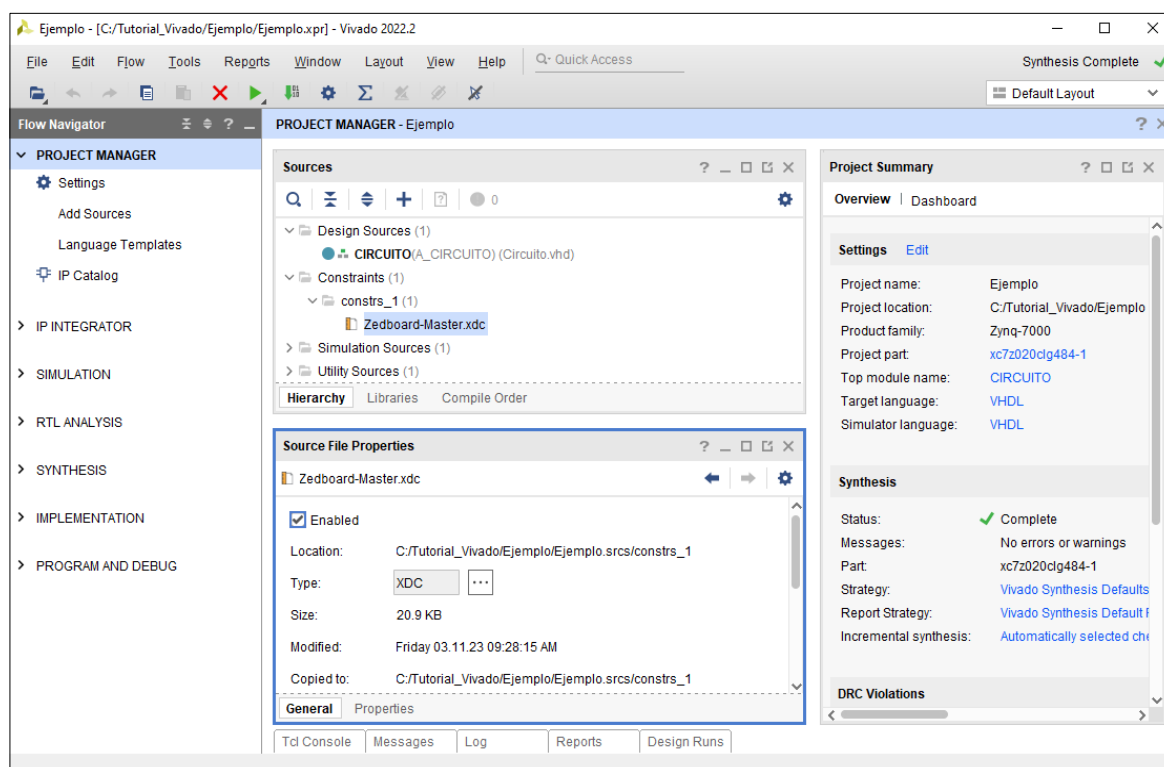


Figura 34. Presentación del fichero de restricciones de la tarjeta ZedBoard en la pantalla principal de Vivado.

Para este ejemplo, vamos a conectar los terminales de nuestro circuito tal como se muestra en la Figura 35. En dicha figura se representan en color rojo los nombres de los pines de la FPGA a los que se encuentran conectados en la tarjeta ZedBoard los conmutadores y leds a

emplear para este ejemplo, y en color azul los nombres de los terminales de nuestro circuito que queremos asociar a estos elementos.

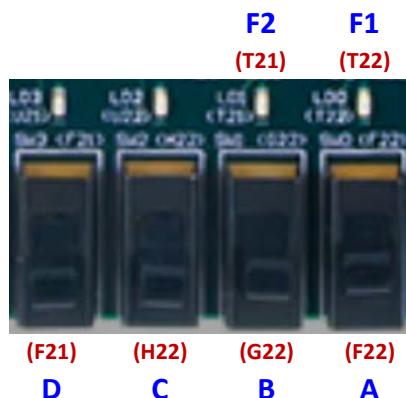


Figura 35. Modo de conexión de los terminales del circuito en la tarjeta de desarrollo ZedBoard.

Para asociar las entradas y salidas de nuestro circuito a los diferentes terminales de la FPGA en la tarjeta de desarrollo, deberemos editar el fichero de restricciones. Para ello, haremos doble click sobre el nombre de dicho fichero, con lo que éste se abrirá en la parte derecha de la pantalla principal de Vivado.

Al editar el fichero, deberemos mantener comentadas (con una # al principio) todas las líneas del mismo, excepto las que se indican a continuación.

Deberemos activar las líneas correspondientes a los LEDs a emplear (**LD0** y **LD1**) quitándoles la # inicial. Además, en estas líneas, en la parte de [get_ports {LDx}], sustituiremos {LD0} por **F1** y {LD1} por **F2**, como se muestra seguidamente:

```
## User LEDs - Bank 33
set_property PACKAGE_PIN T22 [get_ports F1];
set_property PACKAGE_PIN T21 [get_ports F2];
```

Deberemos activar también las líneas correspondientes a los conmutadores a emplear (**SW0**, **SW1**, **SW2** y **SW3**). Además, en estas líneas, en la parte de [get_ports {SWx}], sustituiremos {SW0} por **A**, {SW1} por **B**, {SW2} por **C** y {SW3} por **D**, como se muestra a continuación:

```
## User DIP Switches - Bank 35
set_property PACKAGE_PIN F22 [get_ports A];
set_property PACKAGE_PIN G22 [get_ports B];
set_property PACKAGE_PIN H22 [get_ports C];
set_property PACKAGE_PIN F21 [get_ports D];
```

Por último, deberemos mantener sin comentar las cuatro líneas siguientes:

```
# Note that the bank voltage for IO Bank 33 is fixed to 3.3V on ZedBoard.
set_property IOSTANDARD LVCMOS33 [get_ports -of_objects [get_iobanks 33]];

# Set the bank voltage for IO Bank 34 to 1.8V by default.
set_property IOSTANDARD LVCMOS18 [get_ports -of_objects [get_iobanks 34]];

# Set the bank voltage for IO Bank 35 to 1.8V by default.
set_property IOSTANDARD LVCMOS18 [get_ports -of_objects [get_iobanks 35]];

# Note that the bank voltage for IO Bank 13 is fixed to 3.3V on ZedBoard.
set_property IOSTANDARD LVCMOS33 [get_ports -of_objects [get_iobanks 13]];
```


Una vez editado el fichero de restricciones, ya podemos proceder a la implementación del sistema, para lo cual, en la zona **Flow Navigator** situada en la parte izquierda de la pantalla principal de Vivado, desplegaremos la opción **Implementation** y ejecutaremos a continuación **Run Implementation** (Figura 36).

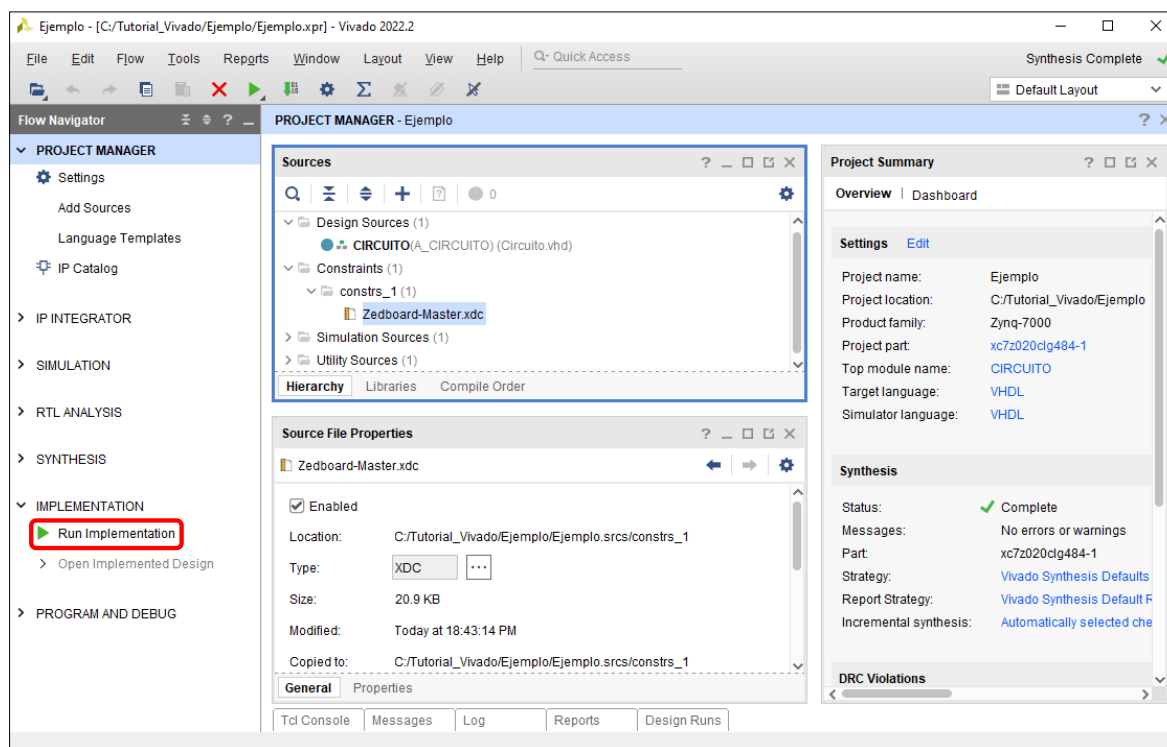


Figura 36. Ejecución de la implementación del diseño.

Si la implementación del diseño concluye sin errores, el programa nos lo indicará mediante una ventana emergente como la mostrada en la Figura 37, que además nos permitirá abrir el diseño implementado, generar el *Bitstream* o visualizar los informes relativos al proceso realizado. Seleccionaremos **View Reports** y haremos click sobre **OK**.

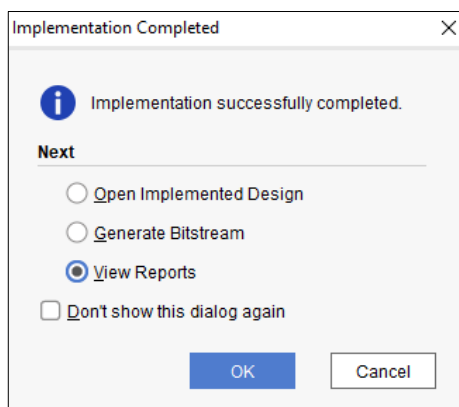


Figura 37. Comunicación de la finalización satisfactoria del proceso de implementación.

7. Programación de la tarjeta de desarrollo.

Para llevar a cabo la programación de la tarjeta de desarrollo deberemos generar previamente el fichero de programación, que recibe el nombre de *Bitstream*.

Para ello, una vez concluida con éxito la implementación del sistema, desplegaremos la opción **Program and Debug** en la parte izquierda de la pantalla principal de Vivado, y a continuación ejecutaremos **Generate Bitstream**, como se muestra en la Figura 38.

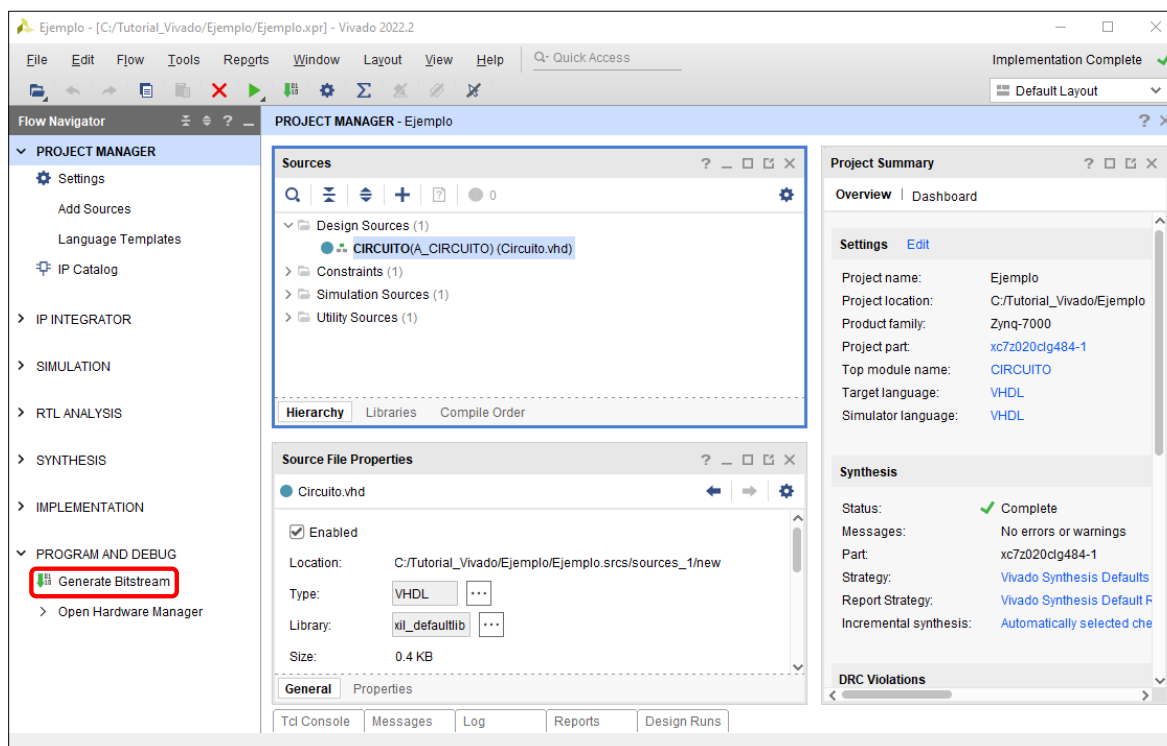


Figura 38. Generación del fichero de programación de la tarjeta (*Bitstream*).

Tras ejecutar la opción **Generate Bitstream** aparecerá la ventana de la Figura 39, donde se nos solicitará la confirmación antes de iniciar la generación del fichero.

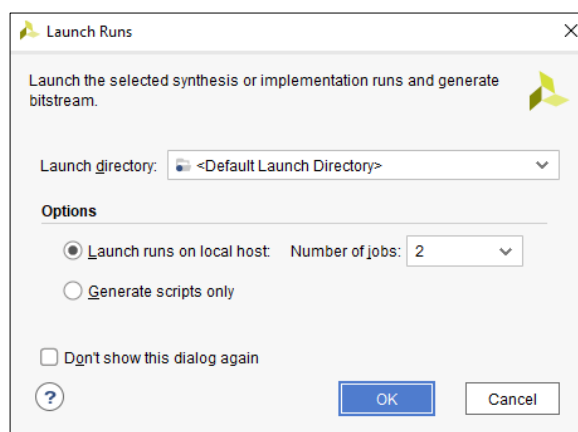


Figura 39. Confirmación de la generación del fichero de programación.

Tras validar con **OK**, después de la aparición de algunas ventanas intermedias similares a las que hemos observado en procesos anteriores, el programa nos informará de que ha terminado la generación del *Bitstream* mediante una ventana como la de la Figura 40. Desde esta ventana podremos, abrir el diseño implementado, visualizar informes sobre el proceso realizado, abrir el gestor del hardware o generar un fichero de configuración para la memoria. En nuestro caso, seleccionaremos la opción **View Reports** y validaremos con **OK**.

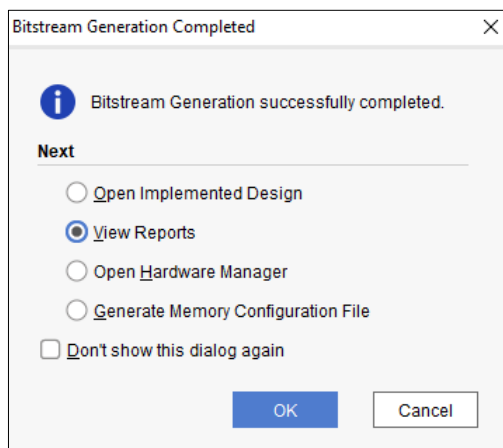


Figura 40. Comunicación de la creación satisfactoria del fichero de restricciones.

Una vez creado el fichero de programación, ya sólo queda programar la tarjeta haciendo uso del mismo. Para ello, conectaremos la tarjeta al ordenador a través del puerto USB, comprobaremos que está encendida y a continuación ejecutaremos la opción **Open Hardware Manager**, situada en la parte izquierda de la pantalla principal de Vivado, como se muestra en la Figura 41.

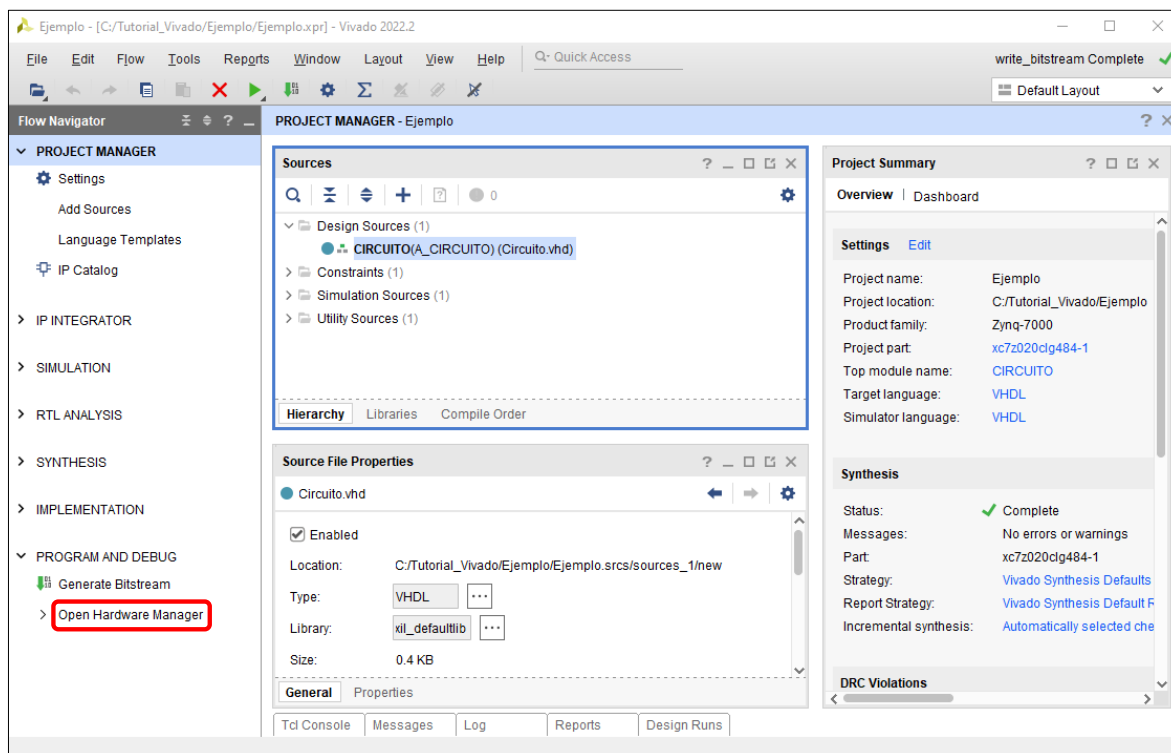


Figura 41. Apertura del gestor del hardware.

A continuación, en la sección **Hardware Manager**, haremos click sobre **Open target** y seguidamente sobre **Auto Connect** para establecer la conexión con la tarjeta de desarrollo, como se muestra en la Figura 42.

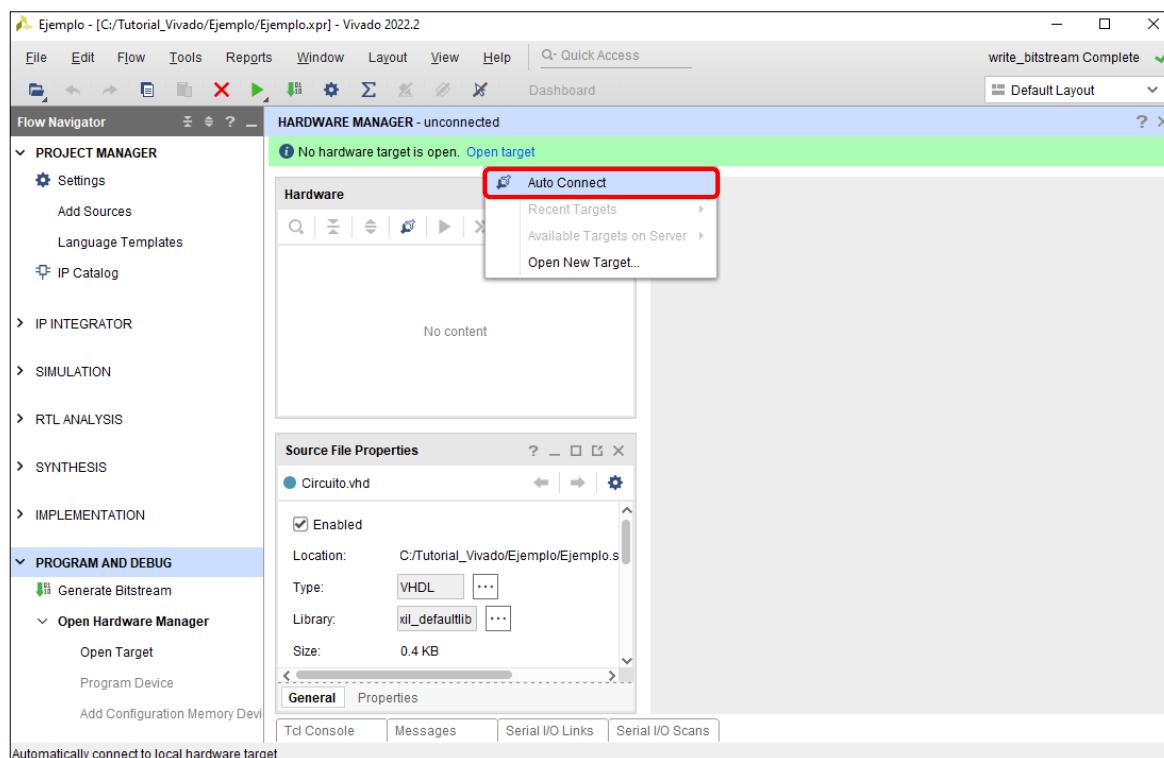


Figura 42. Establecimiento de la conexión con la tarjeta de desarrollo.

Seguidamente, haremos click sobre **Program Device**, como se muestra en la Figura 43.

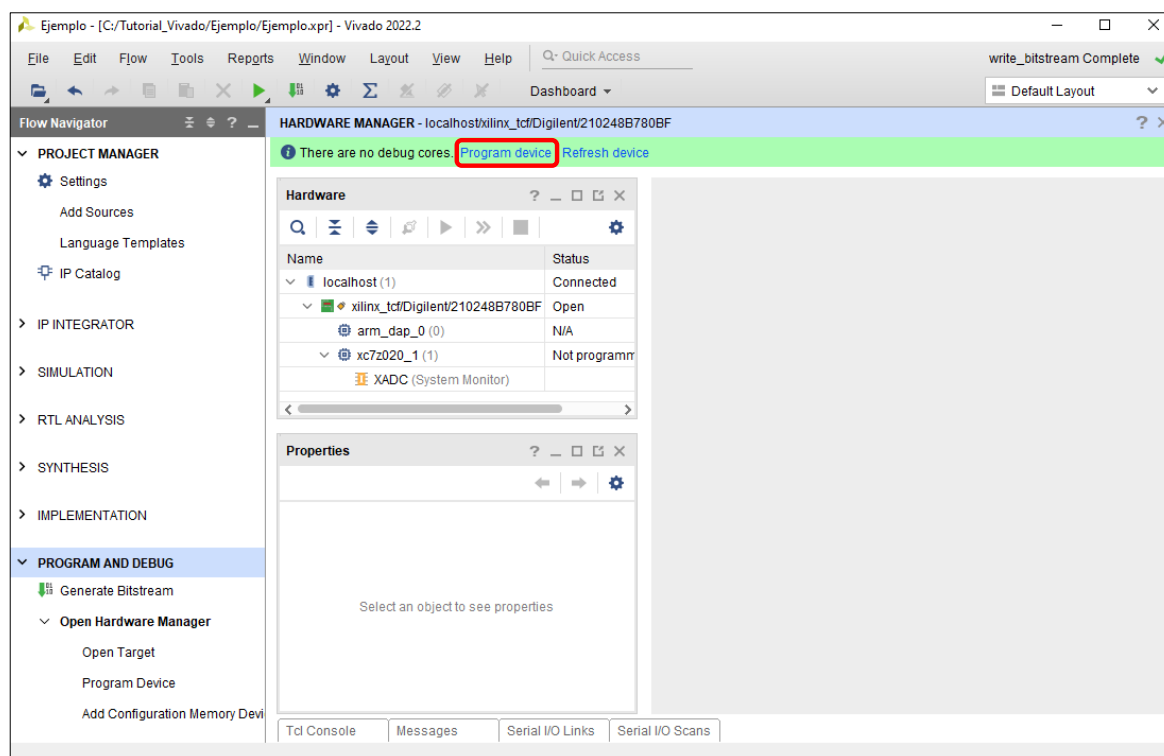


Figura 43. Programación de la tarjeta de desarrollo.

Una vez hecho esto, aparecerá la ventana de la Figura 44, donde el software nos indicará el fichero de programación que va a usar para programar la tarjeta de desarrollo, en este caso **Circuito.bit**.

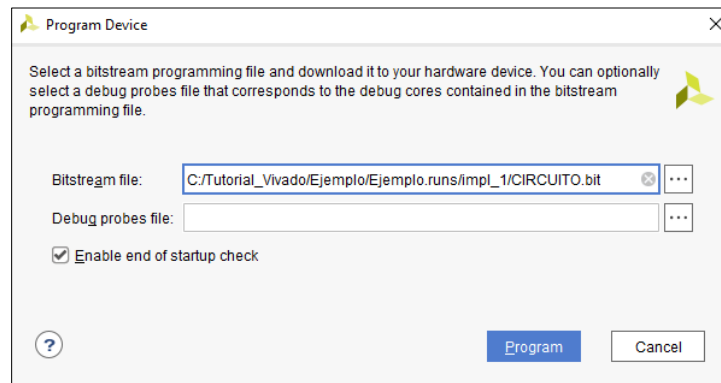


Figura 44. Fichero a emplear para la programación de la tarjeta de desarrollo.

Por último, tras hacer click sobre **Program** se llevará a cabo la programación de la tarjeta.

Una vez finalizada la programación de la tarjeta de desarrollo se nos avisará mediante el encendido del **LED DONE** de dicha tarjeta. A continuación, manipularemos los conmutadores de la misma para aplicar al circuito las diferentes combinaciones de entrada y observaremos el encendido o no encendido de los LEDs para comprobar que los valores de las salidas son los correctos para cada una de ellas.